LAPORAN PROYEK 1

PENERAPAN MODEL MA, EXPONENTIAL SMOOTHING, AR, ARMA, ARIMA, DAN SARIMA PADA DATA DERET WAKTU PM2,5 KOTA SURAKARTA

Mata Kuliah: Analisis Data Time Series (3 SKS)

Disusun oleh:

[Nama Anggota 1] – [NIM Anggota 1]

[Nama Anggota 2] – [NIM Anggota 2]

[Nama Anggota 3] – [NIM Anggota 3]

[Nama Anggota 4] – [NIM Anggota 4]

[Nama Anggota 5] – [NIM Anggota 5]

PROGRAM STUDI SAINS DATA

SEMESTER 5

2026

RINGKASAN

Penelitian ini menganalisis konsentrasi PM2,5 mingguan di Kota Surakarta menggunakan data model CAMS global melalui Open-Meteo Air Quality API pada 8 Agustus 2022–21 Oktober 2024. Sebanyak 116 minggu homogen tanpa nilai hilang dipakai; 104 minggu menjadi data latih dan 12 minggu terakhir data uji. Periode setelah Oktober 2024 dikeluarkan karena patahan rasio PM10/PM2,5. Penelitian menerapkan rata-rata bergerak, SES, Holt, Holt-Winters, AR, ARMA, ARIMA, SARIMA, dan Fourier-ARIMA. Stasioneritas diuji dengan ADF dan KPSS; pemilihan model memakai AICc dan Ljung-Box; evaluasi memakai MAE, RMSE, MAPE, dan MASE. Model terpilih SARIMA(2,1,0)(1,0,0)[52] dengan RMSE uji 4,81 µg/m³, MAE 2,95 µg/m³, MAPE 15,95%, dan MASE 0,380. Namun naive random walk menghasilkan RMSE 4,64 dan uji Diebold-Mariano tidak signifikan (p=0,197), sehingga keunggulan model tidak dapat dibedakan. Peramalan 12 minggu relatif datar sekitar 14–15 µg/m³ dengan interval 80% dan 95% yang melebar. Keterbatasan utama adalah panjang seri hanya 2,2 siklus tahunan, suku musiman tidak signifikan, dan residual heteroskedastik.

BAB 1 PENDAHULUAN

1.1 Latar Belakang

PM2,5 merupakan partikel halus berdiameter aerodinamis tidak lebih dari 2,5 mikrometer yang dapat masuk jauh ke saluran pernapasan dan berkontribusi terhadap penyakit kardiovaskular, pernapasan, serta kematian dini (Cohen et al., 2017). Sebagai kota dengan aktivitas transportasi, perdagangan, dan permukiman yang padat, Surakarta menghadapi tekanan kualitas udara yang perlu dipantau secara berkala. Peramalan konsentrasi PM2,5 bermanfaat untuk memberi peringatan dini, merancang kebijakan, dan mengalokasikan sumber daya kesehatan masyarakat.

Ketersediaan stasiun pengukuran PM2,5 darat di Surakarta terbatas. Oleh karena itu digunakan data sekunder dari model CAMS global yang disediakan melalui Open-Meteo Air Quality API. Data ini merupakan keluaran model reanalisis, bukan pengukuran stasiun, sehingga hasil harus ditafsirkan sebagai representasi grid atmosfer di sekitar koordinat Surakarta (Open-Meteo, 2026; Copernicus Atmosphere Monitoring Service, 2024).

Pemeriksaan awal menemukan patahan struktural pada November 2024: rasio PM10/PM2,5 turun dari sekitar 1,46 menjadi 1,04. Karena rasio tersebut tidak lagi masuk akal secara fisik, periode setelah Oktober 2024 dikeluarkan dari pemodelan. Keputusan ini menjaga homogenitas seri dan menjadi bagian dari kontrol kualitas data.

1.2 Tujuan Analisis

- Menyiapkan dan mendeskripsikan seri mingguan PM2,5 Kota Surakarta dari sumber sekunder yang dapat ditelusuri.
- Mengeksplorasi pola tren, musiman, autokorelasi, dan stasioneritas seri.
- Mengestimasi dan membandingkan model MA, Exponential Smoothing, AR, ARMA, ARIMA, dan SARIMA.
- Mendiagnosis residual serta memilih model dengan kriteria yang valid.
- Meramalkan PM2,5 untuk 12 minggu ke depan beserta interval ketidakpastian 80% dan 95%.

1.3 Manfaat

Secara praktis, hasil peramalan dapat dipakai sebagai skenario awal untuk peringatan dini dan komunikasi risiko. Secara akademik, laporan ini mendokumentasikan alur analisis deret waktu secara lengkap, termasuk keterbatasan data dan asumsi yang dilanggar, sehingga dapat dijalankan ulang dan dikritisi.

BAB 2 DATA DAN EKSPLORASI

2.1 Sumber, Satuan, dan Periode

Data diunduh dari Open-Meteo Air Quality API yang meneruskan keluaran model CAMS global. Koordinat yang dipakai adalah 7,5545° LS dan 110,8060° BT dengan zona waktu Asia/Jakarta. Variabel yang diambil adalah pm2_5 dalam satuan $\mu g/m^3$. Data mentah berbentuk jam-jaman, kemudian diagregasi menjadi rata-rata mingguan. Agregasi dilakukan oleh penulis, bukan oleh penyedia, sehingga bentuk data mentah tetap dapat ditelusuri. Tanggal akses data: 29 Oktober 2026. Ringkasan data dan pembagian latih–uji disajikan pada Tabel 1.

Tautan sumber: https://open-meteo.com/en/docs/air-quality-api dan https://ads.atmosphere.copernicus.eu/datasets/cams-global-reanalysis-eac4. Data CAMS digunakan sebagai representasi kondisi atmosfer; validasi terhadap stasiun darat Surakarta tidak tersedia dalam proyek ini.

Tabel 1. Ringkasan data dan pembagian latih–uji

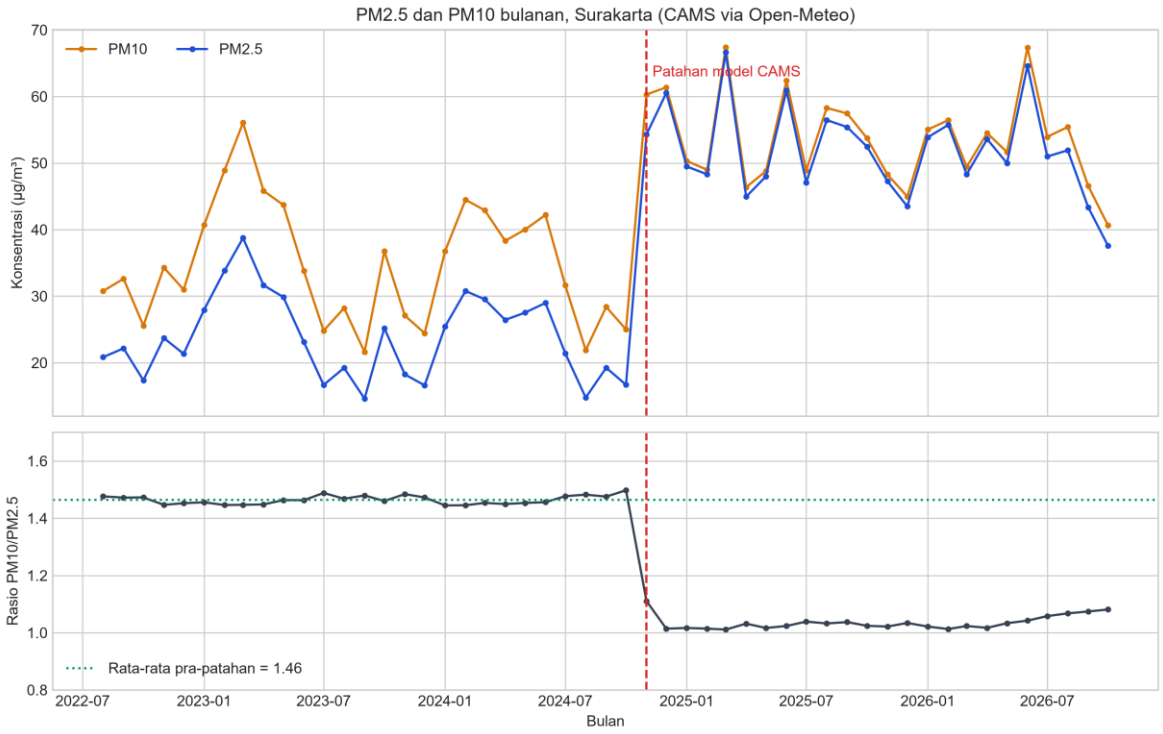
Keterangan	Nilai
Periode model	08-08-2022 – 21-10-2024
Frekuensi	Mingguan (rata-rata 168 jam)
Jumlah observasi	116
Data latih	104 minggu
Data uji	12 minggu
Jam mentah	37.200
Jam valid	37.121
Jam hilang	79 (di awal seri)
Nilai hilang pada seri model	0

Sumber: Open-Meteo/CAMS (2026), diolah penulis.

Dari 37.200 jam mentah, 37.121 jam valid dan 79 jam hilang. Seluruh nilai hilang berada di awal periode (1–4 Agustus 2022) dan tidak berada di dalam rentang model. Setelah agregasi dan penyaringan kualitas, terbentuk 116 minggu lengkap tanpa nilai hilang.

2.2 Patahan Struktural November 2024

Rasio PM10/PM2,5 bulanan dipakai sebagai indikator konsistensi fisik. Sebelum November 2024, rasio rata-rata 1,464 dengan simpangan baku 0,015; sesudahnya rasio turun menjadi 1,036 dengan simpangan baku 0,025. Nilai mendekati 1 berarti PM2,5 hampir sama dengan PM10, yang tidak wajar untuk aerosol atmosfer. Pola serupa ditemukan pada lokasi lain (Jakarta dan Surabaya), sehingga indikasi patahan model CAMS lebih kuat daripada sekadar fluktuasi lokal. Periode setelah 31 Oktober 2024 karena itu tidak dipakai (Gambar 1; Tabel 2).



Gambar 1. Rasio PM10/PM2,5 bulanan dan batas periode homogen

Sumber: Open-Meteo/CAMS (2026), diolah penulis.

Tabel 2. Ringkasan rasio PM10/PM2,5 sebelum dan sesudah patahan

Periode	n bulan	Rata-rata	Simpangan baku	Status
Sebelum Nov 2024	27	1,464	0,015	Homogen

Periode	n bulan	Rata-rata	Simpangan baku	Status
Nov 2024 dan sesudahnya	24	1,036	0,025	Dikeluarkan

Sumber: Open-Meteo/CAMS (2026), diolah penulis.

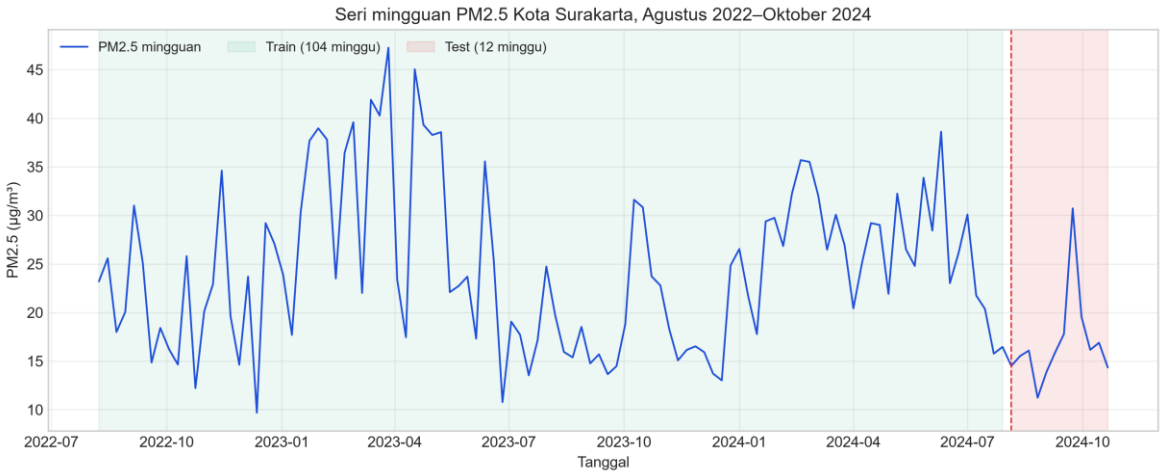
2.3 Statistik Deskriptif

Rata-rata PM2,5 mingguan adalah 23,82 µg/m³ dengan simpangan baku 8,45 µg/m³. Nilai minimum 9,69 µg/m³ dan maksimum 47,26 µg/m³. Distribusi sedikit miring ke kanan (kemencengan 0,635) dengan ekor relatif ringan (kurtosis −0,367). Koefisien variasi 0,355 menunjukkan variasi yang cukup besar terhadap rata-rata (Tabel 3). Pola seri dan pembagian latih–uji ditunjukkan pada Gambar 2.

Tabel 3. Statistik deskriptif seri mingguan PM2,5

Statistik	Nilai
Jumlah observasi	116
Periode awal	08-08-2022
Periode akhir	21-10-2024
Rata-rata	23,82
Median	22,85
Standar deviasi	8,45
Minimum	9,69
Kuartil 1	16,51
Kuartil 3	29,47
Maksimum	47,26
Kemencengan	0,64
Kurtosis	-0,37
Koefisien variasi	0,35
Jumlah hilang	0

Sumber: Open-Meteo/CAMS (2026), diolah penulis.

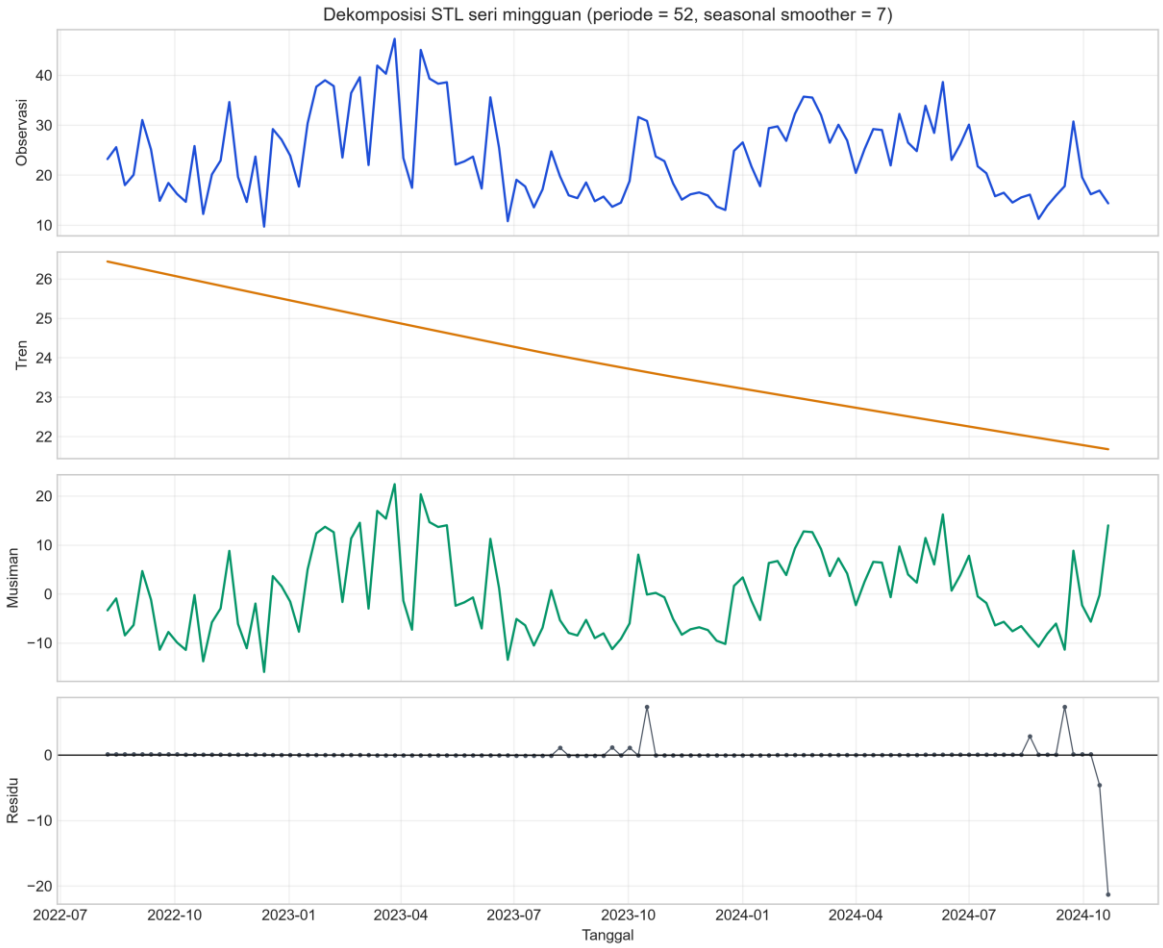


Gambar 2. Seri mingguan PM2,5 Kota Surakarta dan pembagian data latih–uji

Sumber: Open-Meteo/CAMS (2026), diolah penulis.

2.4 Dekomposisi dan Pola Musiman

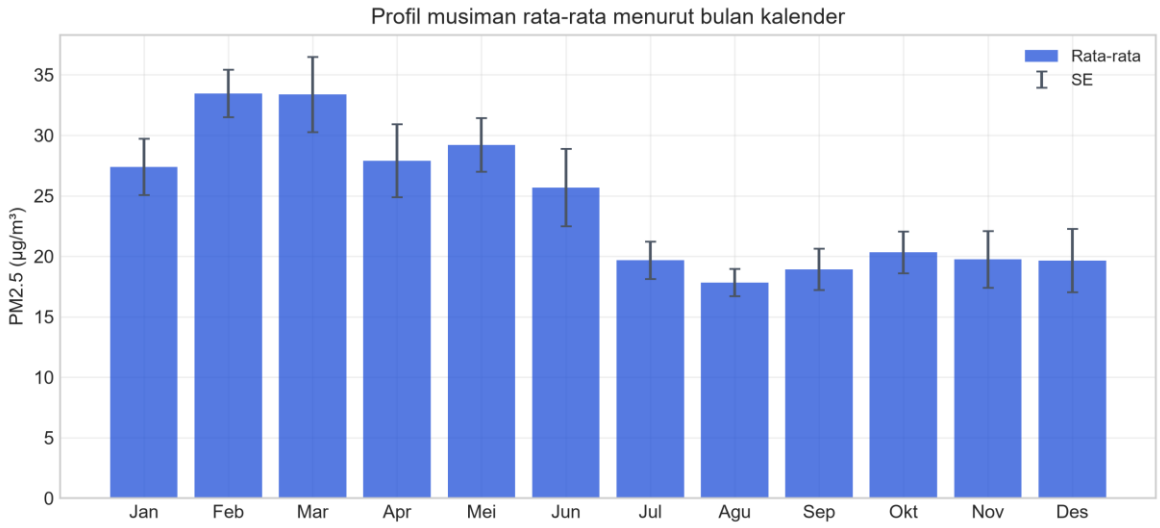
Dekomposisi STL dilakukan dengan periode 52 minggu, seasonal smoother 7, dan sifat robust. Model aditif dipilih karena amplitudo musiman relatif stabil terhadap level dan seri selalu positif. Kekuatan musiman yang dihitung dari rasio varians komponen musiman terhadap seri terdetrend adalah 0,927. Nilai ini tinggi, tetapi harus dibaca hati-hati karena panjang seri hanya sekitar 2,2 siklus tahunan (Gambar 3).



Gambar 3. Dekomposisi STL seri mingguan (periode 52, seasonal smoother 7)

Sumber: Open-Meteo/CAMS (2026), diolah penulis.

Profil bulanan menunjukkan rata-rata lebih tinggi pada Januari–Maret dan lebih rendah pada Juli–September. Pola ini bersifat deskriptif karena setiap bulan hanya terwakili dua sampai tiga kali (Gambar 4). Estimasi komponen musiman tahunan dengan demikian rapuh dan tidak boleh dijadikan satu-satunya dasar kebijakan.

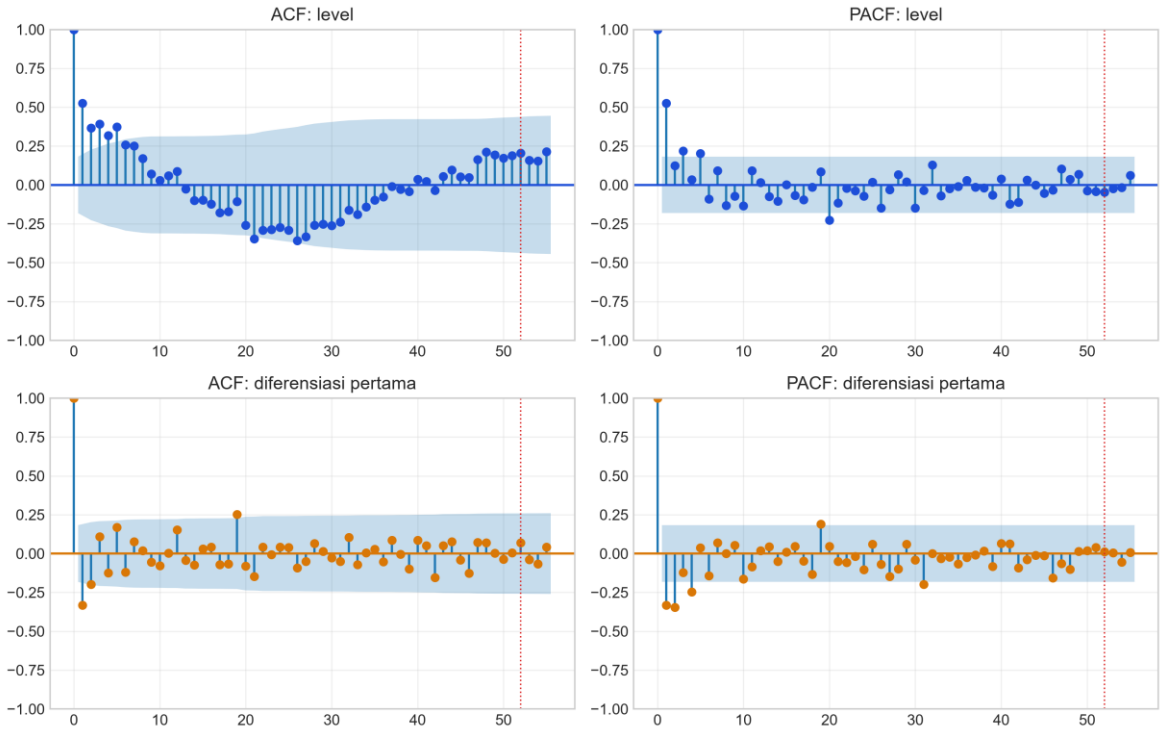


Gambar 4. Profil musiman rata-rata menurut bulan kalender

Sumber: Open-Meteo/CAMS (2026), diolah penulis.

2.5 ACF dan PACF

ACF level meluruh lambat, mengindikasikan ketidakstasioneran. Setelah diferensiasi pertama, ACF menghilang lebih cepat. Pada lag musiman 52 masih terlihat pola, tetapi jumlah siklus yang tersedia terlalu sedikit untuk mengestimasi struktur musiman panjang secara stabil. Karena itu, model musiman diuji sebagai kandidat, bukan diasumsikan pasti benar (Gambar 5).



Gambar 5. ACF dan PACF level serta diferensiasi pertama (lag 1–55)

Sumber: Open-Meteo/CAMS (2026), diolah penulis.

BAB 3 METODE

3.1 Rata-rata Bergerak dan Exponential Smoothing

Rata-rata bergerak sederhana dengan jendela 3 dan 8 minggu dipakai sebagai penghalus dan pembanding naif. Jendela 3 minggu dipilih untuk menangkap perubahan jangka pendek, sedangkan jendela 8 minggu memberi penghalusan lebih kuat. Keduanya bukan model probabilistik sehingga tidak memiliki AIC yang sebanding dengan keluarga ARIMA.

SES meramalkan level dengan pembaruan $\ell_t = \alpha y_t + (1-\alpha)\ell_{t-1}$. Holt menambahkan tren $b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)b_{t-1}$. Holt-Winters menambahkan komponen musiman aditif dengan periode 52. Parameter smoothing dan nilai awal diestimasi dengan maximum likelihood. Komponen tren dan musiman hanya dipertahankan bila memberi perbaikan yang dapat dijelaskan; AIC antar-ES dan ARIMA tidak dibandingkan langsung (Hyndman & Athanasopoulos, 2021).

3.2 AR, MA, ARMA, dan ARIMA

Model AR(p) menyatakan y_t sebagai fungsi linear dari p nilai sebelumnya; MA(q) menyatakan y_t sebagai fungsi linear dari q galat sebelumnya; ARMA(p,q) menggabungkan keduanya. Stasioneritas AR diperiksa dari akar polinomial AR, sedangkan invertibilitas MA diperiksa dari akar polinomial MA. Semua akar harus berada di luar lingkaran satuan (Box et al., 2015; Shumway & Stoffer, 2017).

ARIMA(p,d,q) menambahkan diferensiasi ke-d. Nilai d ditentukan dari ADF/KPSS dan ACF, bukan dari pengulangan differencing sampai grafik tampak rata. Kandidat ARIMA dicari dengan grid sistematis p,q = 0–3 pada d = 1. Pencarian ini berfungsi sebagai pencarian otomatis; hasilnya dikritik dengan membandingkan AICc, Ljung-Box, dan RMSE uji. `pmdarima` tidak dipakai karena lingkungan memakai NumPy 2.4.6 yang tidak kompatibel dengan versi `pmdarima` yang tersedia.

3.3 SARIMA dan Fourier

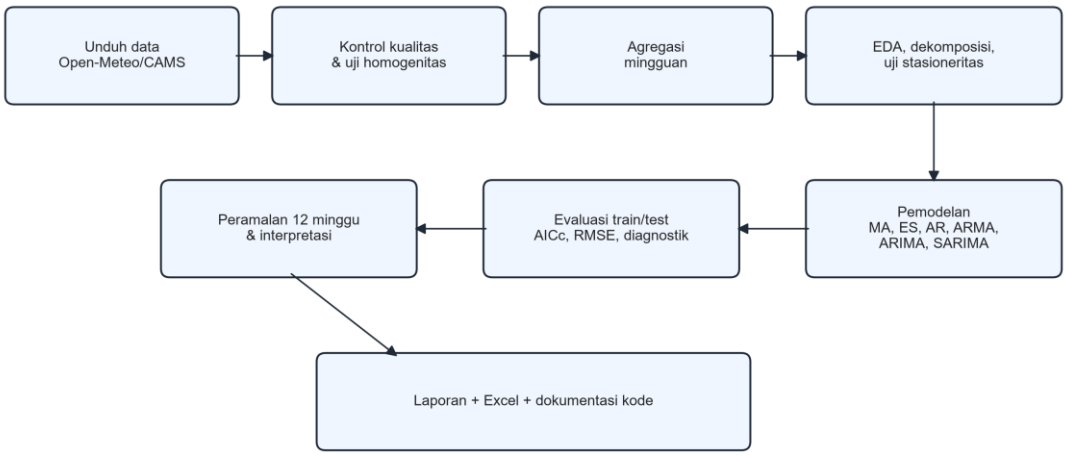
SARIMA(p,d,q)$(P,D,Q)_m$ menambahkan operator musiman. Dengan m = 52 dan hanya 116 minggu, D = 1 akan menghabiskan terlalu banyak observasi; karena itu D = 0 dan struktur musiman ditangkap melalui P,Q. Kandidat dibatasi $p,q,P,Q \in \{0,1,2\}$ dan disaring dengan konvergensi serta Ljung-Box lag 20. Model dengan AICc terendah yang gagal Ljung-Box tidak dipilih.

Sebagai bonus, regresi Fourier dengan K = 1,2,3 harmonik ditambahkan pada ARIMA(1,1,0), ARIMA(1,1,1), dan ARIMA(2,1,0). Regresor Fourier fleksibel, tetapi dapat menghasilkan koefisien MA yang mendekati batas non-invertibilitas; hasilnya dilaporkan sebagai pembanding, bukan model utama.

3.4 Kriteria Evaluasi

Data uji adalah 12 minggu terakhir dan tidak dipakai dalam identifikasi maupun estimasi. Akurasi diukur dengan MAE, RMSE, MAPE, sMAPE, dan MASE. MASE memakai seasonal naive lag 52 pada data latih sebagai skala. AIC/AICc/BIC hanya dibandingkan dalam keluarga ARIMA dengan diferensiasi dan data yang sama. Perbedaan akurasi antar-model diuji dengan Diebold-Mariano. Residual diperiksa dengan ACF, Ljung-Box, Jarque-Bera, Shapiro-Wilk, ARCH LM, dan Durbin-Watson.

Interval peramalan disajikan pada 80% dan 95%. Diagram alur penelitian disajikan pada Gambar 6.



Gambar 6. Diagram alur penelitian

Sumber: Penulis (2026).

BAB 4 HASIL DAN PEMBAHASAN

4.1 Stasioneritas dan Transformasi

Pada level, ADF tidak menolak akar unit (p = 0,203), sedangkan KPSS tidak menolak stasioneritas (p = 0,100). Konflik ini lazim pada sampel pendek. ACF level yang meluruh lambat dan hasil diferensiasi pertama yang sangat signifikan (ADF p < 0,001) mendukung d = 1. Diferensiasi musiman lag 52 juga stasioner, tetapi dengan hanya 63–64 observasi efektif dan 2,2 siklus, D = 1 berisiko over-differencing. Model final memakai d = 1, D = 0, dan komponen AR musiman (Tabel 4).

Transformasi log tidak dipakai karena varians level tidak menunjukkan pembesaran yang konsisten seiring naiknya level, dan transformasi akan menyulitkan interpretasi µg/m³. Heteroskedastisitas residual tetap dideteksi pada tahap diagnostik dan dibahas pada Bagian 4.6.

Tabel 4. Hasil uji ADF dan KPSS

Seri	Uji	Statistik	p-value	Lag	n	Kesimpulan

Seri	Uji	Statistik	p-value	Lag	n	Kesimpulan
Level	ADF	-2,208	0,203	4	111	Tidak stasioner
Level	KPSS	0,161	0,100	5	116	Stasioner
Diferensiasi pertama	ADF	-8,668	<0,001	3	111	Stasioner
Diferensiasi pertama	KPSS	0,288	0,100	47	115	Stasioner
Diferensiasi musiman (lag 52)	ADF	-5,497	<0,001	0	63	Stasioner
Diferensiasi musiman (lag 52)	KPSS	0,104	0,100	3	64	Stasioner

Sumber: Hasil pengolahan penulis (2026).

4.2 Moving Average dan Exponential Smoothing

Tabel 5. Perbandingan moving average dan Exponential Smoothing pada data uji

Model	Spesifikasi	RMSE	MAE	MAPE (%)	MASE
MA(3)	Rata-rata bergerak 3 minggu	4,660	2,941	16,46	0,379
Holt	Trend aditif	5,358	4,544	28,37	0,586
SES	Simple exponential smoothing	5,464	4,732	29,68	0,610
MA(8)	Rata-rata bergerak 8 minggu	6,687	6,162	39,46	0,794
Holt-Winters	Trend + musiman aditif, s=52	6,739	5,186	28,74	0,668

Sumber: Hasil pengolahan penulis (2026).

MA(3) memberi RMSE 4,660, lebih baik daripada MA(8) (6,687). SES dan Holt menghasilkan RMSE sekitar 5,46 dan 5,36; keduanya lebih buruk daripada naive. Holt-Winters memiliki RMSE 6,739 dan gagal Ljung-Box ($p = 0,006$), sehingga tidak layak dipakai. Parameter Holt-Winters juga berada di batas ($\alpha \approx 0$, $\beta = 0$, $\gamma = 0$), tanda bahwa komponen musiman tidak teridentifikasi dengan baik pada seri sependek ini (Tabel 5; Tabel 6).

Tabel 6. Parameter smoothing dan nilai awal model Exponential Smoothing

Model	Parameter	Nilai
SES	smoothing_level	0,3363
SES	smoothing_trend	—
SES	smoothing_seasonal	—
SES	initial_level	22,7236
SES	initial_trend	—

Model	Parameter	Nilai
SES	initial_seasons	vektor kosong
Holt	smoothing_level	0,3367
Holt	smoothing_trend	0
Holt	smoothing_seasonal	—
Holt	initial_level	22,8128
Holt	initial_trend	-0,0297
Holt	initial_seasons	vektor kosong
Holt-Winters	smoothing_level	1,49e-08
Holt-Winters	smoothing_trend	0
Holt-Winters	smoothing_seasonal	0
Holt-Winters	initial_level	26,7604
Holt-Winters	initial_trend	-0,0412
Holt-Winters	initial_seasons	vektor 52 nilai; awal=-4.1601

Sumber: Hasil pengolahan penulis (2026).

4.3 AR, MA, dan ARMA

Tabel 7. Kandidat AR dan ARMA teratas menurut AICc

Model	Diferensiasi	AIC	AICc	BIC
AR(12)	—	641,61	646,33	676,91
AR(11)	—	647,64	651,69	680,57
AR(10)	—	653,33	656,76	683,85
AR(9)	—	659,47	662,34	687,57
AR(8)	—	664,38	666,74	690,02
ARMA(2,3)	d=0	686,83	688,05	705,07
ARMA(1,3)	d=0	687,47	688,38	703,10
ARMA(0,3)	d=0	689,39	690,03	702,41
ARMA(3,3)	d=0	688,64	690,22	709,48
ARMA(2,2)	d=0	691,37	692,26	707,06

Sumber: Hasil pengolahan penulis (2026).

AR(12) terpilih dalam grid AR dengan AICc 646,33, tetapi RMSE ujinya 6,572. ARMA(2,3) terpilih dalam grid ARMA dengan AICc 688,05 dan RMSE 7,067 (Tabel 7). Keduanya stabil: modulus akar AR minimum 1,147 dan modulus akar MA minimum 1,124, sehingga syarat stasioneritas dan invertibilitas terpenuhi (Tabel 8). Meskipun

demikian, akurasi keduanya jauh di bawah naive dan SARIMA, sehingga tidak dipilih sebagai model final.

Tabel 8. Pemeriksaan akar model AR, ARMA, dan SARIMA

Model	Komponen	Jumlah akar	|root| minimum	|root| maksimum	Kesimpulan
AR(12)	AR	12	1,1514	11,8800	Stabil (di luar lingkaran satuan)
ARMA(2,3)	AR	2	1,1472	1,5640	Stabil (di luar lingkaran satuan)
ARMA(2,3)	MA	3	1,1239	9,6479	Stabil (di luar lingkaran satuan)
SARIMA(2,1,0)(1,0,0)[52]	AR	54	1,0442	2,4110	Stabil (di luar lingkaran satuan)

Sumber: Hasil pengolahan penulis (2026).

4.4 ARIMA

Tabel 9. Kandidat ARIMA teratas menurut AICc (d = 1)

p	d	q	AIC	AICc	BIC	Ljung-Box p	Konvergen
1	1	3	679,62	680,27	692,60	0,871	Ya
0	1	3	681,29	681,72	691,68	0,766	Ya
2	1	3	681,62	682,53	697,19	0,872	Ya
3	1	3	683,61	684,84	701,77	0,853	Ya
1	1	2	685,86	686,28	696,28	0,886	Ya

Sumber: Hasil pengolahan penulis (2026).

ARIMA(1,1,3) terpilih dengan AICc 680,27 dan Ljung-Box p = 0,871, tetapi RMSE ujinya 5,708. Kandidat ARIMA(0,1,3) dan ARIMA(2,1,3) memiliki AICc yang berdekatan, sehingga perbedaan di antara mereka tipis (Tabel 9). Pencarian otomatis ini menunjukkan bahwa kriteria informasi saja tidak cukup: model ARIMA non-musiman tetap kalah dari naive pada data uji.

4.5 SARIMA

Tabel 10. Kandidat SARIMA teratas menurut AICc

p	d	q	P	D	Q	s	AICc	Ljung-Box p	Konvergen
0	1	2	0	0	1	52	306,91	0,007	Ya
1	1	2	0	0	1	52	307,12	0,006	Ya

p	d	q	P	D	Q	s	AICc	Ljung-Box p	Konvergen
0	1	2	1	0	1	52	307,69	0,005	Ya
1	1	2	1	0	1	52	308,62	0,004	Ya
2	1	2	0	0	1	52	309,66	0,003	Ya
2	1	0	1	0	0	52	310,63	0,081	Ya
0	1	1	0	0	1	52	310,69	0,006	Ya

Sumber: Hasil pengolahan penulis (2026).

Tiga kandidat dengan AICc terendah memakai Q = 1 dan memiliki Ljung-Box p < 0,01, sehingga residualnya masih mengandung autokorelasi. Model SARIMA(2,1,0)(1,0,0)[52] berada pada peringkat berikutnya dengan AICc 310,63 dan Ljung-Box p = 0,081. Model ini dipilih karena merupakan kandidat ber-AICc rendah yang lolos penyaringan residual, bukan karena AICc terendah secara mutlak (Tabel 10). Kandidat dengan D = 1 diuji terpisah dan tidak memberi perbaikan yang cukup untuk mengimbangi hilangnya observasi.

Persamaan model final dapat dituliskan sebagai $(1 - \varphi_1 B - \varphi_2 B^2)(1 - \Phi_1 B^{52})(1 - B)Y_t = \varepsilon_t$, dengan $\varphi_1 = -0{,}317$, $\varphi_2 = -0{,}191$, dan $\Phi_1 = 0{,}0737$. Koefisien AR(1) signifikan pada $\alpha = 5\%$ ($p = 0{,}015$), sedangkan AR(2) ($p = 0{,}238$) dan AR musiman ($p = 0{,}409$) tidak signifikan. Akibatnya, struktur musiman yang diestimasi lemah dan tidak boleh ditafsirkan sebagai bukti kuat adanya siklus tahunan.

4.6 Diagnostik Residual

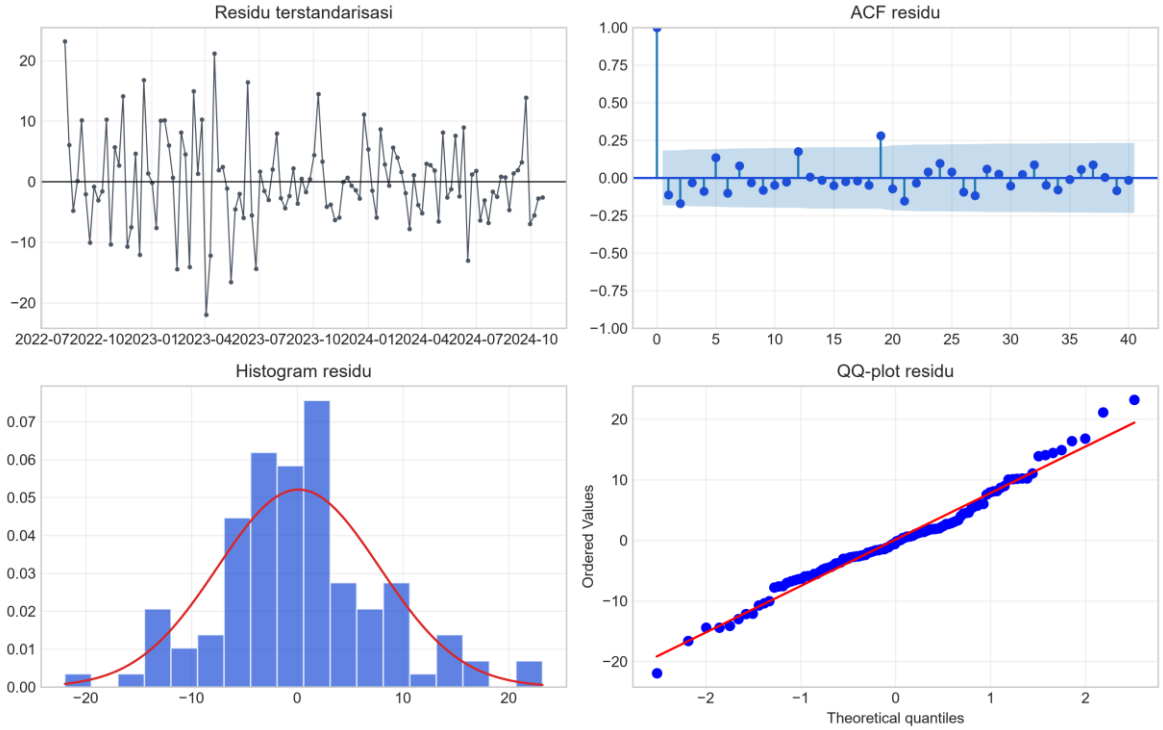
Tabel 11. Diagnostik residual model final

Uji	Statistik	p-value	Interpretasi
Ljung-Box lag 10	11,669	0,308	Tidak ada autokorelasi
Ljung-Box lag 20	28,559	0,097	Tidak ada autokorelasi
Ljung-Box lag 52	58,601	0,246	Tidak ada autokorelasi
Jarque-Bera	4,266	0,119	Normal
ARCH LM (5 lag)	12,919	0,024	Heteroskedastik
Shapiro-Wilk	0,981	0,097	Normal
Durbin-Watson	2,146	—	Mendekati 2 = baik

Sumber: Hasil pengolahan penulis (2026).

Ljung-Box lag 10, 20, dan 52 tidak menolak tidak adanya autokorelasi (p = 0,308; 0,097; 0,246). Jarque-Bera (p = 0,119) dan Shapiro-Wilk (p = 0,097) tidak

menolak normalitas. Durbin-Watson 2,146 mendekati 2 (Tabel 11; Gambar 7). Namun ARCH LM menolak homoskedastisitas (p = 0,024), sehingga varians residual berubah-ubah. Konsekuensinya, interval peramalan yang dihitung dengan asumsi varians konstan cenderung terlalu sempit pada periode bergejolak dan terlalu lebar pada periode tenang.



Gambar 7. Diagnostik residual model SARIMA final

Sumber: Hasil pengolahan penulis (2026).

4.7 Perbandingan Model

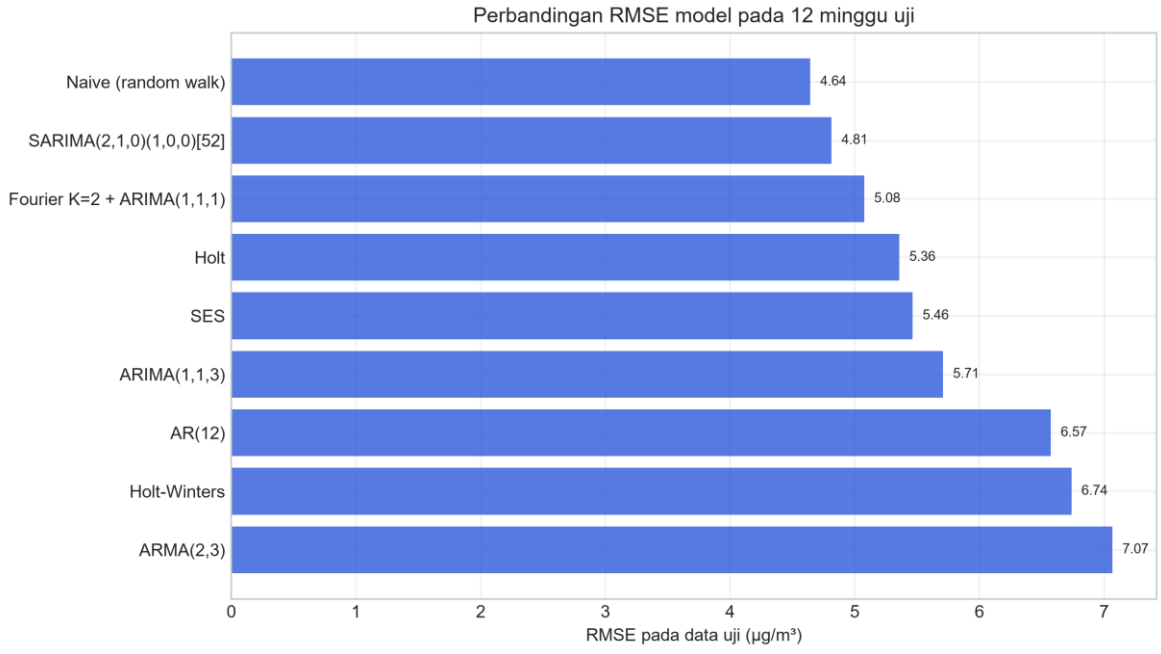
Tabel 12. Perbandingan seluruh model pada data uji

Model	AICc	RMSE	MAE	MAPE (%)	MASE	LB20 p	AIC sebanding
Naive (random walk)	—	4,643	2,769	14,98	0,357	—	Tidak
MA(3)	—	4,660	2,941	16,46	0,379	—	Tidak
SARIMA(2,1,0)(1,0,0)[52]	310,63	4,814	2,951	15,95	0,380	0,081	Ya
Fourier K=2 + ARIMA(1,1,1)	685,59	5,076	4,125	25,32	0,532	0,534	Ya
Holt	423,85	5,358	4,544	28,37	0,586	—	Tidak
SES	419,86	5,464	4,732	29,68	0,610	—	Tidak
ARIMA(1,1,3)	680,27	5,708	5,085	32,15	0,655	0,871	Ya
AR(12)	646,33	6,572	6,187	39,47	0,797	0,994	Ya

Model	AICc	RMSE	MAE	MAPE (%)	MASE	LB20 p	AIC sebanding
MA(8)	—	6,687	6,162	39,46	0,794	—	Tidak
Holt-Winters	430,81	6,739	5,186	28,74	0,668	—	Tidak
ARMA(2,3)	688,05	7,067	6,795	43,74	0,876	0,863	Ya
Seasonal naive	—	8,585	6,223	36,63	0,802	—	Tidak

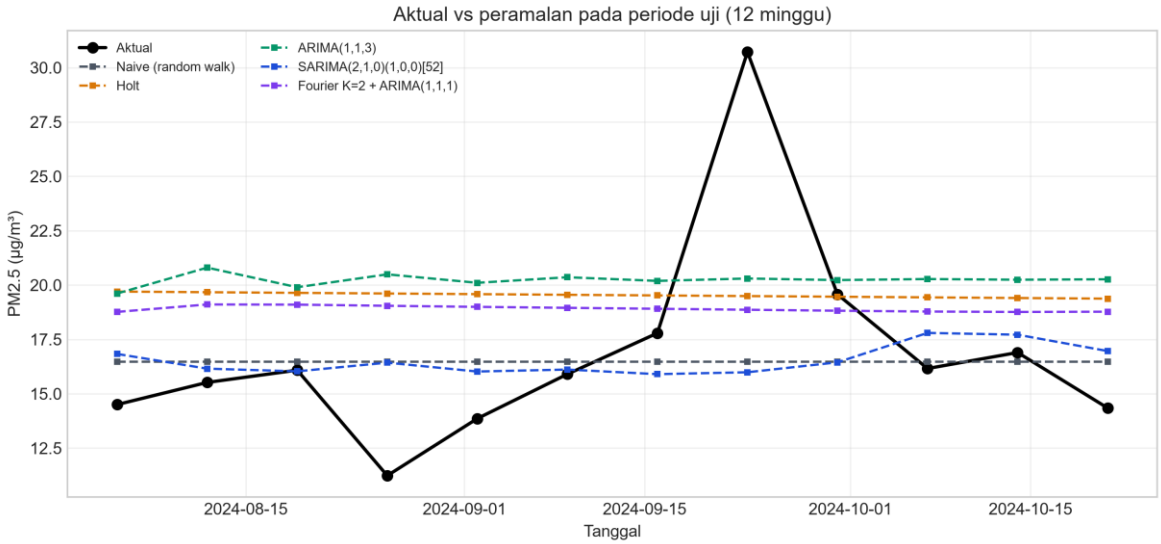
Sumber: Hasil pengolahan penulis (2026). AICc hanya sebanding untuk model ARIMA dengan d dan data yang sama.

Naive random walk memiliki RMSE 4,643 dan MAE 2,769, sedikit lebih baik daripada SARIMA (RMSE 4,814; MAE 2,951). MA(3) berada di antara keduanya (RMSE 4,660). Perbedaan SARIMA dan naive tidak signifikan secara statistik (Diebold-Mariano p = 0,197), sehingga keduanya tidak dapat dibedakan secara meyakinkan. SARIMA tetap dipilih karena memberi struktur musiman dan interval peramalan eksplisit, bukan karena unggul akurasi (Tabel 12; Gambar 8; Gambar 9; Tabel 13).



Gambar 8. Perbandingan RMSE model pada 12 minggu uji

Sumber: Hasil pengolahan penulis (2026).



Gambar 9. Aktual versus peramalan beberapa model pada periode uji

Sumber: Hasil pengolahan penulis (2026).

Tabel 13. Uji Diebold-Mariano model final terhadap pembanding utama

Model pembanding	DM statistik	p-value	Kesimpulan
Naive (random walk)	1,375	0,197	Tidak berbeda signifikan
MA(3)	0,632	0,540	Tidak berbeda signifikan
Fourier K=2 + ARIMA(1,1,1)	-0,329	0,748	Tidak berbeda signifikan
Holt	-0,576	0,576	Tidak berbeda signifikan
ARIMA(1,1,3)	-0,799	0,441	Tidak berbeda signifikan

Sumber: Hasil pengolahan penulis (2026).

4.8 Peramalan dan Interpretasi

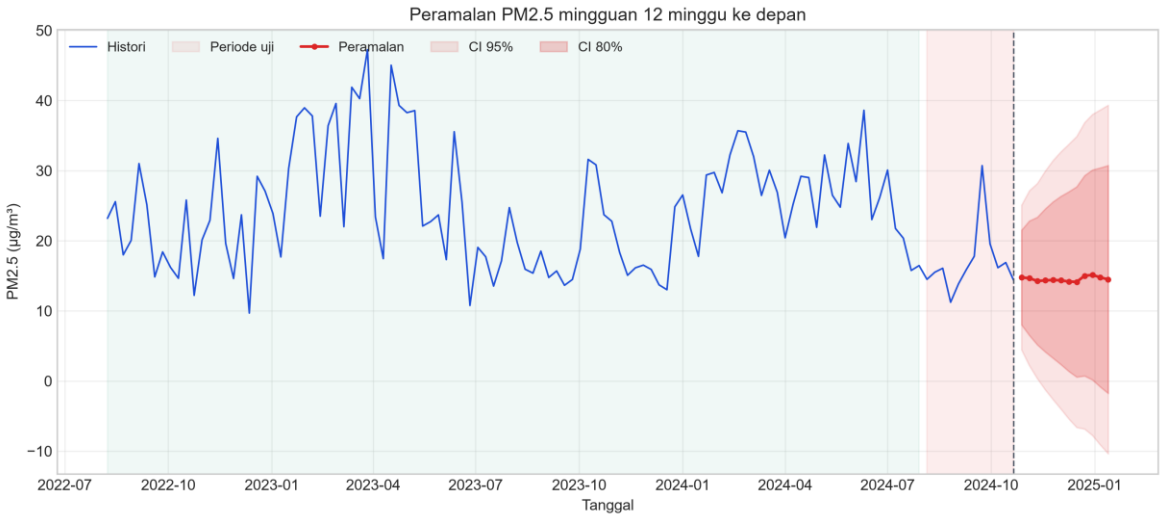
Tabel 14. Peramalan PM2,5 12 minggu ke depan dengan interval 80% dan 95%

Minggu	Peramalan	Bawah 80%	Atas 80%	Bawah 95%	Atas 95%
28-10-2024	14,77	8,02	21,52	4,45	25,09
04-11-2024	14,67	6,50	22,84	2,17	27,17
11-11-2024	14,26	5,17	23,36	0,35	28,18
18-11-2024	14,35	4,16	24,55	-1,24	29,94
25-11-2024	14,41	3,25	25,56	-2,65	31,47
02-12-2024	14,35	2,35	26,35	-4,00	32,71
09-12-2024	14,19	1,38	27,00	-5,40	33,78
16-12-2024	14,14	0,57	27,71	-6,62	34,90
23-12-2024	15,01	0,72	29,30	-6,84	36,86

Minggu	Peramalan	Bawah 80%	Atas 80%	Bawah 95%	Atas 95%
30-12-2024	15,13	0,16	30,11	-7,76	38,03
06-01-2025	14,78	-0,85	30,41	-9,12	38,68
13-01-2025	14,49	-1,77	30,75	-10,37	39,35

Sumber: Hasil pengolahan penulis (2026); satuan µg/m³.

Peramalan bergerak datar dari 14,77 µg/m³ pada minggu pertama menjadi 14,49 µg/m³ pada minggu terakhir, dengan rata-rata 14,55 µg/m³. Interval 80% melebar dari 8,02–21,52 menjadi −1,77–30,75; interval 95% melebar dari 4,45–25,09 menjadi −10,37–39,35. Batas bawah negatif adalah artefak aproksimasi normal dan tidak memiliki makna fisik. Secara substantif, model memprediksi stabilisasi pada tingkat sedang, tetapi ketidakpastiannya besar sehingga angka ini lebih tepat dibaca sebagai skenario (Tabel 14; Gambar 10).



Gambar 10. Peramalan PM2,5 12 minggu ke depan dengan interval 80% dan 95%

Sumber: Hasil pengolahan penulis (2026).

BAB 5 KESIMPULAN

5.1 Kesimpulan

- Data PM2,5 mingguan Kota Surakarta yang homogen mencakup 116 minggu (8 Agustus 2022–21 Oktober 2024) tanpa nilai hilang; 104 minggu dipakai untuk latih dan 12 minggu untuk uji.

- Patahan rasio PM10/PM2,5 pada November 2024 terdeteksi dan periode sesudahnya dikeluarkan agar seri tetap konsisten secara fisik.
- Uji ADF/KPSS dan ACF/PACF mendukung diferensiasi pertama (d = 1); D = 0 dipilih untuk menghindari over-differencing musiman.
- Model terpilih adalah SARIMA(2,1,0)(1,0,0)[52] dengan AICc latih 310,63 dan RMSE uji 4,814 µg/m³.
- Naive random walk memiliki RMSE 4,643; uji Diebold-Mariano tidak signifikan (p = 0,197), sehingga keunggulan SARIMA tidak dapat dibedakan secara meyakinkan.
- Peramalan 12 minggu relatif datar sekitar 14–15 µg/m³, dengan interval 80% dan 95% yang melebar cepat.

5.2 Keterbatasan

- Data berasal dari grid model CAMS, bukan stasiun pengukuran darat Surakarta, sehingga tidak menangkap variasi lokal skala halus.
- Panjang seri hanya 116 minggu atau sekitar 2,2 siklus tahunan; estimasi komponen musiman rapuh dan suku AR musiman tidak signifikan.
- Residual model final heteroskedastik (ARCH LM p = 0,024), sehingga interval peramalan dengan varians konstan tidak sepenuhnya valid.
- Tidak ada validasi silang bergulir (rolling-origin); evaluasi hanya memakai satu periode uji 12 minggu.
- Model ARIMA/SARIMA mengasumsikan hubungan linear; potensi efek libur, kebakaran lahan, dan perubahan kebijakan belum dimasukkan sebagai variabel eksogen.

5.3 Saran

Untuk penelitian lanjutan, disarankan membandingkan data CAMS dengan stasiun darat bila tersedia, menambah variabel eksogen seperti curah hujan, kecepatan angin, dan titik panas, serta memakai validasi bergulir. Alternatif model varians bersyarat (misalnya GARCH pada residual) dapat dipertimbangkan untuk mengatasi heteroskedastisitas. Untuk keperluan kebijakan, peramalan sebaiknya disajikan sebagai rentang skenario, bukan angka tunggal.

DAFTAR PUSTAKA

Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). Time series analysis: Forecasting and control (5th ed.). Wiley.

Cohen, A. J., Brauer, M., Burnett, R., Anderson, H. R., Frostad, J., Estep, K., ... Forouzanfar, M. H. (2017). Estimates and 25-year trends of the global burden of disease attributable to ambient air pollution: An analysis of data from the Global Burden of Diseases Study 2015. The Lancet, 389(10082), 1907–1918. https://doi.org/10.1016/S0140-6736(17)30505-6

Copernicus Atmosphere Monitoring Service. (2024). CAMS global reanalysis (EAC4). ECMWF. https://ads.atmosphere.copernicus.eu/datasets/cams-global-reanalysis-eac4

Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and practice (3rd ed.). OTexts. https://otexts.com/fpp3/

Open-Meteo. (2026). Air quality API documentation. https://open-meteo.com/en/docs/air-quality-api

Shumway, R. H., & Stoffer, D. S. (2017). Time series analysis and its applications: With R examples (4th ed.). Springer.

Wei, W. W. S. (2006). Time series analysis: Univariate and multivariate methods (2nd ed.). Pearson.

LAMPIRAN A. PEMBAGIAN PERAN ANGGOTA

Tabel 15. Pembagian peran anggota kelompok

Peran	Anggota	Tugas utama
Data dan eksplorasi	[Nama Anggota 1]	Unduh data, kontrol kualitas, EDA, dekomposisi
Exponential Smoothing	[Nama Anggota 2]	MA, SES, Holt, Holt-Winters
Keluarga ARIMA	[Nama Anggota 3]	AR, ARMA, ARIMA, SARIMA, Fourier
Evaluasi dan diagnostik	[Nama Anggota 4]	Uji residual, perbandingan, DM
Penulisan dan reproducibility	[Nama Anggota 5]	Laporan, dokumentasi, reproduksi

Sumber: Penulis (2026).

LAMPIRAN B. KODE LENGKAP

Kode berikut adalah versi yang dipakai untuk menghasilkan seluruh output. Pipeline utama dijalankan berurutan: fetch_openmeteo.py, preprocess.py, check_homogeneity.py, lalu analysis.py. build_report.py menghasilkan dokumen ini. Notebook walkthrough tersedia di notebooks/01_analisis_pm25_surakarta.ipynb.

B.1 fetch_openmeteo.py

```
"""Unduh PM2.5 historis Surakarta dari Open-Meteo Air Quality API.

Sumber data : Copernicus Atmosphere Monitoring Service (CAMS) melalui Open-Meteo.
Tanpa API key. Dokumentasi: https://open-meteo.com/en/docs/air-quality-api
"""

from __future__ import annotations

import csv
import json
import urllib.parse
import urllib.request
from datetime import date
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
RAW_DIR = ROOT / "data" / "raw"
OUT_DIR = ROOT / "data" / "processed"

LATITUDE = -7.5545
LONGITUDE = 110.8060
TIMEZONE = "Asia/Jakarta"
START_DATE = "2022-08-01"
END_DATE = date.today().isoformat()
HOURLY_VARIABLE = "pm2_5"


def build_url() -> str:
    params = {
        "latitude": LATITUDE,
        "longitude": LONGITUDE,
        "hourly": HOURLY_VARIABLE,
        "start_date": START_DATE,
        "end_date": END_DATE,
        "timezone": TIMEZONE,
    }
    return "https://air-quality-api.open-meteo.com/v1/air-quality?" + urllib.parse.urlencode(params)


def main() -> None:
    RAW_DIR.mkdir(parents=True, exist_ok=True)
    url = build_url()
    print(f"Mengunduh: {url}")

    with urllib.request.urlopen(url, timeout=120) as response:
        payload = json.load(response)

    json_path = RAW_DIR / "openmeteo_pm25_surakarta_hourly.json"
    with json_path.open("w", encoding="utf-8") as handle:
        json.dump(payload, handle, ensure_ascii=False, indent=2)

    times = payload["hourly"]["time"]
    values = payload["hourly"][HOURLY_VARIABLE]

    csv_path = RAW_DIR / "openmeteo_pm25_surakarta_hourly.csv"
    with csv_path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.writer(handle)
        writer.writerow(["datetime", "pm2_5"])
```

```
        for timestamp, value in zip(times, values):
            writer.writerow([timestamp, "" if value is None else value])

    valid = [(t, v) for t, v in zip(times, values) if v is not None]
    missing = len(times) - len(valid)

    OUT_DIR.mkdir(parents=True, exist_ok=True)
    clean_path = OUT_DIR / "pm25_surakarta_hourly_clean.csv"
    with clean_path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.writer(handle)
        writer.writerow(["datetime", "pm2_5"])
        writer.writerows(valid)

    print(f"Lokasi        : {payload.get('latitude')}, {payload.get('longitude')}")
    print(f"Zona waktu    : {payload.get('timezone')}")
    print(f"Satuan        : {payload.get('hourly_units', {}).get(HOURLY_VARIABLE)}")
    print(f"Total jam     : {len(times)}")
    print(f"Jam valid     : {len(valid)}")
    print(f"Jam hilang    : {missing} ({missing / len(times):.2%})")
    if valid:
        print(f"Periode valid : {valid[0][0]} s.d. {valid[-1][0]}")
        print(f"Nilai pertama : {valid[0][0]} = {valid[0][1]} ug/m3")
    print(f"Raw JSON      : {json_path}")
    print(f"Raw CSV       : {csv_path}")
    print(f"CSV bersih    : {clean_path}")


if __name__ == "__main__":
    main()
```

B.2 preprocess.py

```
"""Bersihkan data hourly Open-Meteo menjadi series harian, mingguan, dan bulanan.

Output:
- data/processed/pm25_surakarta_daily.csv
- data/processed/pm25_surakarta_weekly.csv
- data/processed/pm25_surakarta_monthly.csv
- data/processed/pm25_surakarta_monthly_model.csv
"""

from __future__ import annotations

import calendar
import csv
from collections import defaultdict
from datetime import datetime
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
RAW_CSV = ROOT / "data" / "raw" / "openmeteo_pm25_surakarta_hourly.csv"
OUT_DIR = ROOT / "data" / "processed"


def load_hourly() -> pd.DataFrame:
    frame = pd.read_csv(RAW_CSV)
    frame["datetime"] = pd.to_datetime(frame["datetime"])
    frame["pm2_5"] = pd.to_numeric(frame["pm2_5"], errors="coerce")
    frame = frame.dropna(subset=["datetime"]).sort_values("datetime")
    return frame


def aggregate(frame: pd.DataFrame, keys: list[str]) -> pd.DataFrame:
    grouped = (
        frame.groupby(keys, dropna=False)["pm2_5"]
        .agg(pm25_mean="mean", pm25_median="median", pm25_min="min", pm25_max="max", n_hours="count")
        .reset_index()
    )
    return grouped


def main() -> None:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    hourly = load_hourly()

    valid = hourly.dropna(subset=["pm2_5"])
    print(f"Hourly valid: {len(valid)} / {len(hourly)}")
    print(f"Periode     : {valid['datetime'].min()} s.d. {valid['datetime'].max()}")

    hourly["date"] = hourly["datetime"].dt.date
    daily = aggregate(hourly, ["date"])
    daily["date"] = pd.to_datetime(daily["date"])
    daily["expected_hours"] = 24
    daily["coverage"] = daily["n_hours"] / daily["expected_hours"]
    daily = daily.sort_values("date").reset_index(drop=True)
    daily.to_csv(OUT_DIR / "pm25_surakarta_daily.csv", index=False)

    hourly["month"] = hourly["datetime"].dt.to_period("M").astype(str)
    monthly = aggregate(hourly, ["month"])
    monthly["expected_hours"] = monthly["month"].map(
        lambda value: calendar.monthrange(int(value[:4]), int(value[5:7]))[1] * 24
    )
    monthly["coverage"] = monthly["n_hours"] / monthly["expected_hours"]
    monthly["complete"] = monthly["coverage"] >= 0.95
    monthly = monthly.sort_values("month").reset_index(drop=True)
    monthly.to_csv(OUT_DIR / "pm25_surakarta_monthly.csv", index=False)

    hourly["week_start"] = (
        hourly["datetime"] - pd.to_timedelta(hourly["datetime"].dt.weekday, unit="D")
    ).dt.normalize()
    weekly = aggregate(hourly, ["week_start"])
    weekly["week_start"] = pd.to_datetime(weekly["week_start"])
    weekly["expected_hours"] = 7 * 24
```

```
    weekly["coverage"] = weekly["n_hours"] / weekly["expected_hours"]
    weekly["complete"] = weekly["coverage"] >= 0.95
    weekly = weekly.sort_values("week_start").reset_index(drop=True)
    weekly.to_csv(OUT_DIR / "pm25_surakarta_weekly.csv", index=False)

    break_start = pd.Timestamp("2024-11-01")
    monthly["homogeneous"] = monthly["month"] < break_start.strftime("%Y-%m")
    weekly["homogeneous"] = (weekly["week_start"] + pd.Timedelta(days=6)) < break_start
    monthly.to_csv(OUT_DIR / "pm25_surakarta_monthly.csv", index=False)
    weekly.to_csv(OUT_DIR / "pm25_surakarta_weekly.csv", index=False)

    monthly_model = monthly.loc[
        monthly["complete"] & monthly["homogeneous"], ["month", "pm25_mean", "coverage"]
    ].rename(columns={"pm25_mean": "pm25"})
    monthly_model.to_csv(OUT_DIR / "pm25_surakarta_monthly_model.csv", index=False)

    weekly_model = weekly.loc[
        weekly["complete"] & weekly["homogeneous"],
        ["week_start", "pm25_mean", "n_hours", "coverage"],
    ].rename(columns={"pm25_mean": "pm25"})
    weekly_model = weekly_model.sort_values("week_start").reset_index(drop=True)

    expected_weeks = pd.date_range(
        weekly_model["week_start"].min(), weekly_model["week_start"].max(), freq="7D"
    )
    missing_weeks = expected_weeks.difference(weekly_model["week_start"])
    if len(missing_weeks):
        print(f"Minggu hilang di dalam rentang: {list(missing_weeks)}")
        weekly_model = (
            weekly_model.set_index("week_start")
            .reindex(expected_weeks)
            .rename_axis("week_start")
            .reset_index()
        )
        weekly_model["pm25"] = weekly_model["pm25"].interpolate(method="linear")
    weekly_model.to_csv(OUT_DIR / "pm25_surakarta_weekly_model.csv", index=False)

    print("\nRingkasan bulanan:")
    print(monthly[["month", "pm25_mean", "coverage", "complete"]].to_string(index=False))
    print("\nRingkasan series homogen (sebelum patahan Nov 2024):")
    print(f"Bulan lengkap : {len(monthly_model)} (tidak cukup untuk model bulanan)")
    print(f"Minggu lengkap: {len(weekly_model)}")
    print(
        f"Rentang minggu: {weekly_model['week_start'].min().date()} s.d. "
        f"{weekly_model['week_start'].max().date()}"
    )
    print(f"Minggu hilang : {len(missing_weeks)}")
    print(f"\nOutput: {OUT_DIR}")


if __name__ == "__main__":
    main()
```

B.3 check_homogeneity.py

```
"""Periksa homogenitas seri PM2.5 Open-Meteo/CAMS memakai rasio PM2.5/PM10.

Perubahan basis model CAMS pada November 2024 membuat PM2.5 hampir sama
dengan PM10. Seri setelah patahan tersebut tidak dipakai untuk pemodelan.
"""

from __future__ import annotations

import json
import urllib.parse
import urllib.request
from datetime import date
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
RAW_DIR = ROOT / "data" / "raw"
OUT_DIR = ROOT / "data" / "processed"
FIG_DIR = ROOT / "outputs" / "figures"

LATITUDE = -7.5545
LONGITUDE = 110.8060
TIMEZONE = "Asia/Jakarta"
START_DATE = "2022-08-01"
END_DATE = date.today().isoformat()


def build_url(variables: str) -> str:
    params = {
        "latitude": LATITUDE,
        "longitude": LONGITUDE,
        "hourly": variables,
        "start_date": START_DATE,
        "end_date": END_DATE,
        "timezone": TIMEZONE,
    }
    return "https://air-quality-api.open-meteo.com/v1/air-quality?" + urllib.parse.urlencode(params)


def main() -> None:
    RAW_DIR.mkdir(parents=True, exist_ok=True)
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    FIG_DIR.mkdir(parents=True, exist_ok=True)

    url = build_url("pm10,pm2_5")
    print(f"Mengunduh: {url}")
    with urllib.request.urlopen(url, timeout=120) as response:
        payload = json.load(response)

    raw_path = RAW_DIR / "openmeteo_pm10_pm25_surakarta_hourly.json"
    with raw_path.open("w", encoding="utf-8") as handle:
        json.dump(payload, handle, ensure_ascii=False)

    frame = pd.DataFrame(
        {
            "datetime": pd.to_datetime(payload["hourly"]["time"]),
            "pm10": payload["hourly"]["pm10"],
            "pm25": payload["hourly"]["pm2_5"],
        }
    )
    frame["month"] = frame["datetime"].dt.to_period("M").astype(str)
    monthly = (
        frame.groupby("month")[["pm10", "pm25"]]
        .mean()
        .reset_index()
        .rename(columns={"pm10": "pm10_mean", "pm25": "pm25_mean"})
    )
    monthly["ratio"] = monthly["pm10_mean"] / monthly["pm25_mean"]
    monthly["homogeneous"] = monthly["month"] < "2024-11"
    monthly.to_csv(OUT_DIR / "pm25_pm10_monthly.csv", index=False)
```

```
    pre = monthly.loc[monthly["homogeneous"], "ratio"]
    print("\nRasio PM10/PM2.5 sebelum Nov 2024:")
    print(pre.describe().round(3).to_string())
    print("\nRasio PM10/PM2.5 sesudah Nov 2024:")
    print(monthly.loc[~monthly["homogeneous"], "ratio"].describe().round(3).to_string())

    plt.style.use("seaborn-v0_8-whitegrid")
    fig, axes = plt.subplots(2, 1, figsize=(11, 7), sharex=True, height_ratios=[3, 2])
    x = pd.to_datetime(monthly["month"])

    axes[0].plot(x, monthly["pm10_mean"], color="#d97706", marker="o", ms=3, label="PM10")
    axes[0].plot(x, monthly["pm25_mean"], color="#1d4ed8", marker="o", ms=3, label="PM2.5")
    axes[0].axvline(pd.Timestamp("2024-11-01"), color="#dc2626", linestyle="--", linewidth=1.5)
    axes[0].text(
        pd.Timestamp("2024-11-10"),
        axes[0].get_ylim()[1] * 0.9,
        "Patahan model CAMS",
        color="#dc2626",
        fontsize=10,
    )
    axes[0].set_ylabel("Konsentrasi (µg/m³)")
    axes[0].set_title("PM2.5 dan PM10 bulanan, Surakarta (CAMS via Open-Meteo)")
    axes[0].legend(loc="upper left", ncol=2)

    axes[1].plot(x, monthly["ratio"], color="#374151", marker="o", ms=3)
    axes[1].axhline(pre.mean(), color="#059669", linestyle=":", linewidth=1.5, label=f"Rata-rata pra-
patahan = {pre.mean():.2f}")
    axes[1].axvline(pd.Timestamp("2024-11-01"), color="#dc2626", linestyle="--", linewidth=1.5)
    axes[1].set_ylabel("Rasio PM10/PM2.5")
    axes[1].set_xlabel("Bulan")
    axes[1].set_ylim(0.8, 1.7)
    axes[1].legend(loc="lower left")

    fig.tight_layout()
    fig.savefig(FIG_DIR / "fig01_data_break.png", dpi=300, bbox_inches="tight")
    plt.close(fig)

    print(f"\nTabel : {OUT_DIR / 'pm25_pm10_monthly.csv'}")
    print(f"Gambar: {FIG_DIR / 'fig01_data_break.png'}")


if __name__ == "__main__":
    main()
```

B.4 analysis.py

```
"""Analisis deret waktu PM2.5 mingguan Surakarta (seri homogen 2022-08 s.d. 2024-10).

Metode: MA smoothing, SES, Holt, Holt-Winters, AR, ARMA, ARIMA, SARIMA,
serta Fourier + ARIMA sebagai bonus. Evaluasi memakai split train/test
104/12 minggu dan peramalan 12 minggu ke depan.
"""

from __future__ import annotations

import json
import warnings
from dataclasses import dataclass
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
from scipy import stats
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox, het_arch
from statsmodels.stats.stattools import durbin_watson, jarque_bera
from statsmodels.tsa.ar_model import AutoReg
from statsmodels.tsa.holtwinters import ExponentialSmoothing, SimpleExpSmoothing
from statsmodels.tsa.seasonal import STL
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.stattools import adfuller, kpss

warnings.filterwarnings("ignore")

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data" / "processed"
OUT = ROOT / "outputs"
FIG = OUT / "figures"
TAB = OUT / "tables"
for folder in (OUT, FIG, TAB):
    folder.mkdir(parents=True, exist_ok=True)

SEASON = 52
TEST_SIZE = 12
HORIZON = 12
FINAL_NAME = "SARIMA(2,1,0)(1,0,0)[52]"
FINAL_ORDER = (2, 1, 0)
FINAL_SEASONAL = (1, 0, 0, SEASON)

plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams.update(
    {
        "figure.dpi": 120,
        "savefig.dpi": 300,
        "font.size": 10,
        "axes.titlesize": 12,
        "axes.labelsize": 10,
        "legend.fontsize": 9,
        "axes.grid": True,
        "grid.alpha": 0.3,
    }
)
COLORS = {
    "blue": "#1d4ed8",
    "orange": "#d97706",
    "red": "#dc2626",
    "green": "#059669",
    "gray": "#4b5563",
    "purple": "#7c3aed",
}


def to_py(value):
    if isinstance(value, (np.integer,)):
        return int(value)
    if isinstance(value, (np.floating,)):
        return float(value)
```

```
    if isinstance(value, (np.ndarray,)):
        return value.tolist()
    if isinstance(value, (pd.Timestamp,)):
        return value.strftime("%Y-%m-%d")
    return value


def load_series() -> tuple[pd.DataFrame, pd.Series]:
    frame = pd.read_csv(DATA / "pm25_surakarta_weekly_model.csv", parse_dates=["week_start"])
    frame = frame.sort_values("week_start").reset_index(drop=True)
    series = frame.set_index("week_start")["pm25"].asfreq("7D")
    if series.isna().any():
        raise ValueError("Series mingguan masih memiliki nilai hilang.")
    return frame, series


def score(y_true, y_pred, train, season: int = SEASON) -> dict:
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    mae = float(np.mean(np.abs(y_true - y_pred)))
    rmse = float(np.sqrt(np.mean((y_true - y_pred) ** 2)))
    mape = float(np.mean(np.abs((y_true - y_pred) / y_true)) * 100)
    smape = float(np.mean(2 * np.abs(y_true - y_pred) / (np.abs(y_true) + np.abs(y_pred))) * 100)
    scale = float(np.mean(np.abs(train.values[season:] - train.values[:-season])))
    mase = float(mae / scale) if scale > 0 else np.nan
    return {"MAE": mae, "RMSE": rmse, "MAPE": mape, "sMAPE": smape, "MASE": mase}


def aicc_from_aic(aic: float, k: int, n: int) -> float:
    if aic is None or not np.isfinite(aic) or n - k - 1 <= 0:
        return np.nan
    return float(aic + 2 * k * (k + 1) / (n - k - 1))


def moving_average_forecast(train: pd.Series, window: int, horizon: int) -> np.ndarray:
    history = list(train.values.astype(float))
    forecasts = []
    for _ in range(horizon):
        value = float(np.mean(history[-window:]))
        forecasts.append(value)
        history.append(value)
    return np.asarray(forecasts)


def fit_sarimax(train, order, seasonal_order, trend, exog=None, exog_forecast=None):
    model = SARIMAX(
        train,
        exog=exog,
        order=order,
        seasonal_order=seasonal_order,
        trend=trend,
        enforce_stationarity=False,
        enforce_invertibility=False,
    )
    fit = model.fit(disp=False)
    forecast = fit.forecast(horizon, exog=exog_forecast)
    return fit, np.asarray(forecast)


def lb_pvalue(resid, lag: int = 20):
    try:
        return float(acorr_ljungbox(resid, lags=[lag], return_df=True)["lb_pvalue"].iloc[0])
    except Exception:
        return np.nan


def record(results, name, spec, fit, forecast, train, test, k, aic, aicc, bic, note="",
comparable=True):
    metrics = score(test.values, forecast, train)
    resid = getattr(fit, "resid", None)
    results.append(
        {
            "Model": name,
            "Spesifikasi": spec,
            "k": k,
            "AIC": aic,
```

```
            "AICc": aicc,
            "BIC": bic,
            **metrics,
            "LB20_p": lb_pvalue(resid) if resid is not None else np.nan,
            "Konvergen": note or "Ya",
            "AIC_sebanding": "Ya" if comparable else "Tidak",
            "_forecast": np.asarray(forecast, dtype=float),
        }
    )


def root_summary(name: str, roots, kind: str) -> dict | None:
    values = np.asarray(roots, dtype=complex)
    if values.size == 0:
        return None
    modulus = np.abs(values)
    stable = bool(np.all(modulus > 1.0))
    return {
        "Model": name,
        "Komponen": kind,
        "Jumlah akar": int(values.size),
        "|root| minimum": float(modulus.min()),
        "|root| maksimum": float(modulus.max()),
        "Kesimpulan": "Stabil (di luar lingkaran satuan)" if stable else "Tidak stabil",
    }


def es_param_rows(name: str, fit) -> list[dict]:
    rows = []
    for key, value in fit.params.items():
        if value is None:
            rows.append({"Model": name, "Parameter": key, "Nilai": "tidak diestimasi"})
            continue
        array = np.asarray(value)
        if array.ndim == 0:
            displayed = float(array)
        else:
            flat = array.ravel()
            displayed = f"vektor {flat.size} nilai; awal={flat[0]:.4f}" if flat.size else "vektor
kosong"
        rows.append({"Model": name, "Parameter": key, "Nilai": displayed})
    return rows


def build_flowchart() -> None:
    fig, ax = plt.subplots(figsize=(10, 5.2))
    ax.set_xlim(0, 10)
    ax.set_ylim(0, 5.2)
    ax.axis("off")
    boxes = [
        (0.3, 3.9, 2.0, 0.8, "Unduh data\nOpen-Meteo/CAMS"),
        (2.7, 3.9, 2.0, 0.8, "Kontrol kualitas\n& uji homogenitas"),
        (5.1, 3.9, 2.0, 0.8, "Agregasi\nmingguan"),
        (7.5, 3.9, 2.2, 0.8, "EDA, dekomposisi,\nuji stasioneritas"),
        (7.5, 2.3, 2.2, 0.8, "Pemodelan\nMA, ES, AR, ARMA,\nARIMA, SARIMA"),
        (4.6, 2.3, 2.2, 0.8, "Evaluasi train/test\nAICc, RMSE, diagnostik"),
        (1.7, 2.3, 2.2, 0.8, "Peramalan 12 minggu\n& interpretasi"),
        (3.1, 0.7, 3.8, 0.8, "Laporan + Excel + dokumentasi kode"),
    ]
    for x, y, w, h, text in boxes:
        box = FancyBboxPatch(
            (x, y), w, h, boxstyle="round,pad=0.05,rounding_size=0.08",
            linewidth=1.2, edgecolor="#1f2937", facecolor="#eff6ff",
        )
        ax.add_patch(box)
        ax.text(x + w / 2, y + h / 2, text, ha="center", va="center", fontsize=9)
    arrows = [
        ((2.3, 4.3), (2.7, 4.3)),
        ((4.7, 4.3), (5.1, 4.3)),
        ((7.1, 4.3), (7.5, 4.3)),
        ((8.6, 3.9), (8.6, 3.1)),
        ((7.5, 2.7), (6.8, 2.7)),
        ((4.6, 2.7), (3.9, 2.7)),
        ((2.8, 2.3), (3.8, 1.5)),
    ]
    for start, end in arrows:
```

```
        ax.add_patch(FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=14, color="#1f2937"))
    fig.tight_layout()
    fig.savefig(FIG / "fig00_flowchart.png", bbox_inches="tight")
    plt.close(fig)


def descriptive(series: pd.Series) -> pd.DataFrame:
    data = {
        "Jumlah observasi": len(series),
        "Periode awal": series.index.min().strftime("%Y-%m-%d"),
        "Periode akhir": series.index.max().strftime("%Y-%m-%d"),
        "Rata-rata": series.mean(),
        "Median": series.median(),
        "Standar deviasi": series.std(),
        "Minimum": series.min(),
        "Kuartil 1": series.quantile(0.25),
        "Kuartil 3": series.quantile(0.75),
        "Maksimum": series.max(),
        "Kemencengan": series.skew(),
        "Kurtosis": series.kurtosis(),
        "Koefisien variasi": series.std() / series.mean(),
        "Jumlah hilang": int(series.isna().sum()),
    }
    return pd.DataFrame(list(data.items()), columns=["Statistik", "Nilai"])


def stationarity_table(series: pd.Series) -> pd.DataFrame:
    rows = []
    for label, values in [
        ("Level", series),
        ("Diferensiasi pertama", series.diff().dropna()),
        ("Diferensiasi musiman (lag 52)", series.diff(SEASON).dropna()),
    ]:
        adf = adfuller(values, autolag="AIC")
        rows.append(
            {
                "Seri": label,
                "Uji": "ADF",
                "Statistik": adf[0],
                "p-value": adf[1],
                "Lag": adf[2],
                "n": adf[3],
                "Kesimpulan": "Stasioner" if adf[1] < 0.05 else "Tidak stasioner",
            }
        )
        try:
            kp = kpss(values, regression="c", nlags="auto")
            rows.append(
                {
                    "Seri": label,
                    "Uji": "KPSS",
                    "Statistik": kp[0],
                    "p-value": kp[1],
                    "Lag": kp[2],
                    "n": len(values),
                    "Kesimpulan": "Tidak stasioner" if kp[1] < 0.05 else "Stasioner",
                }
            )
        except Exception:
            pass
    return pd.DataFrame(rows)


def plot_series(series: pd.Series, train: pd.Series, test: pd.Series) -> None:
    fig, ax = plt.subplots(figsize=(11, 4.6))
    ax.plot(series.index, series.values, color=COLORS["blue"], linewidth=1.2, label="PM2.5 mingguan")
    ax.axvspan(train.index[0], train.index[-1], color=COLORS["green"], alpha=0.07, label="Train (104
minggu)")
    ax.axvspan(test.index[0], test.index[-1], color=COLORS["red"], alpha=0.10, label="Test (12
minggu)")
    ax.axvline(test.index[0], color=COLORS["red"], linestyle="--", linewidth=1)
    ax.set_title("Seri mingguan PM2.5 Kota Surakarta, Agustus 2022–Oktober 2024")
    ax.set_ylabel("PM2.5 (µg/m³)")
    ax.set_xlabel("Tanggal")
    ax.legend(loc="upper left", ncol=3)
    fig.tight_layout()
```

```
    fig.savefig(FIG / "fig02_series.png", bbox_inches="tight")
    plt.close(fig)


def plot_decomposition(series: pd.Series) -> float:
    stl = STL(series, period=SEASON, seasonal=7, robust=True).fit()
    fig, axes = plt.subplots(4, 1, figsize=(11, 9), sharex=True)
    axes[0].plot(series.index, series.values, color=COLORS["blue"])
    axes[0].set_ylabel("Observasi")
    axes[1].plot(series.index, stl.trend, color=COLORS["orange"])
    axes[1].set_ylabel("Tren")
    axes[2].plot(series.index, stl.seasonal, color=COLORS["green"])
    axes[2].set_ylabel("Musiman")
    axes[3].plot(series.index, stl.resid, color=COLORS["gray"], marker="o", ms=2, linewidth=0.8)
    axes[3].axhline(0, color="black", linewidth=0.8)
    axes[3].set_ylabel("Residu")
    axes[3].set_xlabel("Tanggal")
    axes[0].set_title("Dekomposisi STL seri mingguan (periode = 52, seasonal smoother = 7)")
    fig.tight_layout()
    fig.savefig(FIG / "fig03_decomposition.png", bbox_inches="tight")
    plt.close(fig)

    detrended = series - stl.trend
    strength = max(0.0, 1 - np.var(stl.resid) / np.var(detrended)) if np.var(detrended) > 0 else np.nan
    return float(strength)


def plot_seasonal_profile(series: pd.Series) -> None:
    profile = series.groupby(series.index.month).agg(["mean", "std", "count"])
    fig, ax = plt.subplots(figsize=(9, 4.2))
    ax.bar(profile.index, profile["mean"], color=COLORS["blue"], alpha=0.75, label="Rata-rata")
    ax.errorbar(
        profile.index,
        profile["mean"],
        yerr=profile["std"] / np.sqrt(profile["count"]),
        fmt="none",
        ecolor=COLORS["gray"],
        capsize=3,
        label="SE",
    )
    ax.set_xticks(range(1, 13))
    ax.set_xticklabels(["Jan", "Feb", "Mar", "Apr", "Mei", "Jun", "Jul", "Agu", "Sep", "Okt", "Nov",
"Des"])
    ax.set_ylabel("PM2.5 (µg/m³)")
    ax.set_title("Profil musiman rata-rata menurut bulan kalender")
    ax.legend()
    fig.tight_layout()
    fig.savefig(FIG / "fig04_seasonal_profile.png", bbox_inches="tight")
    plt.close(fig)


def plot_acf_pacf(series: pd.Series) -> None:
    fig, axes = plt.subplots(2, 2, figsize=(11, 7))
    plot_acf(series, lags=55, ax=axes[0, 0], color=COLORS["blue"])
    axes[0, 0].set_title("ACF: level")
    axes[0, 0].axvline(SEASON, color=COLORS["red"], linestyle=":", linewidth=1)
    plot_pacf(series, lags=55, ax=axes[0, 1], method="ywm", color=COLORS["blue"])
    axes[0, 1].set_title("PACF: level")
    axes[0, 1].axvline(SEASON, color=COLORS["red"], linestyle=":", linewidth=1)
    plot_acf(series.diff().dropna(), lags=55, ax=axes[1, 0], color=COLORS["orange"])
    axes[1, 0].set_title("ACF: diferensiasi pertama")
    axes[1, 0].axvline(SEASON, color=COLORS["red"], linestyle=":", linewidth=1)
    plot_pacf(series.diff().dropna(), lags=55, ax=axes[1, 1], method="ywm", color=COLORS["orange"])
    axes[1, 1].set_title("PACF: diferensiasi pertama")
    axes[1, 1].axvline(SEASON, color=COLORS["red"], linestyle=":", linewidth=1)
    fig.tight_layout()
    fig.savefig(FIG / "fig05_acf_pacf.png", bbox_inches="tight")
    plt.close(fig)


def fit_all_models(train: pd.Series, test: pd.Series) -> tuple[pd.DataFrame, dict, pd.DataFrame,
pd.DataFrame, pd.DataFrame, dict]:
    results = []
    grids = {}
    fitted = {}
    es_rows = []
```

```
    root_rows = []

    naive = np.repeat(train.values[-1], HORIZON)
    record(results, "Naive (random walk)", "ŷ_{t+h} = y_t", None, naive, train, test, 0, np.nan, np.nan,
np.nan, "Benchmark", False)
    fitted["Naive (random walk)"] = naive

    seasonal_naive = train.values[-SEASON:][:HORIZON] if len(train) >= SEASON else
np.repeat(train.mean(), HORIZON)
    record(results, "Seasonal naive", "ŷ_{t+h} = y_{t+h-s2}", None, seasonal_naive, train, test, 0, np.nan,
np.nan, np.nan, "Benchmark", False)
    fitted["Seasonal naive"] = seasonal_naive

    for window in (3, 8):
        pred = moving_average_forecast(train, window, HORIZON)
        record(results, f"MA({window})", f"Rata-rata bergerak {window} minggu", None, pred, train,
test, 1, np.nan, np.nan, np.nan, "Smoothing", False)
        fitted[f"MA({window})"] = pred

    ses = SimpleExpSmoothing(train, initialization_method="estimated").fit(optimized=True)
    pred = np.asarray(ses.forecast(HORIZON))
    k_es = len(getattr(ses, "params", {})) if hasattr(getattr(ses, "params", {}), "__len__") else 2
    record(results, "SES", "Simple exponential smoothing", ses, pred, train, test, k_es, ses.aic,
aicc_from_aic(ses.aic, k_es, len(train)), np.nan, "Smoothing", False)
    fitted["SES"] = pred
    es_rows.extend(es_param_rows("SES", ses))

    holt = ExponentialSmoothing(train, trend="add",
initialization_method="estimated").fit(optimized=True)
    pred = np.asarray(holt.forecast(HORIZON))
    k_holt = len(getattr(holt, "params", {})) if hasattr(getattr(holt, "params", {}), "__len__") else 3
    record(results, "Holt", "Trend aditif", holt, pred, train, test, k_holt, holt.aic,
aicc_from_aic(holt.aic, k_holt, len(train)), np.nan, "Smoothing", False)
    fitted["Holt"] = pred
    es_rows.extend(es_param_rows("Holt", holt))

    hw = ExponentialSmoothing(
        train, trend="add", seasonal="add", seasonal_periods=SEASON, initialization_method="estimated"
    ).fit(optimized=True)
    pred = np.asarray(hw.forecast(HORIZON))
    k_hw = len(getattr(hw, "params", {})) if hasattr(getattr(hw, "params", {}), "__len__") else 4
    record(results, "Holt-Winters", "Trend + musiman aditif, s=52", hw, pred, train, test, k_hw,
hw.aic, aicc_from_aic(hw.aic, k_hw, len(train)), np.nan, "Smoothing", False)
    fitted["Holt-Winters"] = pred
    es_rows.extend(es_param_rows("Holt-Winters", hw))

    best_ar = None
    ar_rows = []
    for p in range(1, 13):
        try:
            fit = AutoReg(train, lags=p, trend="c").fit()
            k = p + 2
            aicc = aicc_from_aic(fit.aic, k, len(train))
            ar_rows.append({"p": p, "AIC": fit.aic, "AICc": aicc, "BIC": fit.bic})
            if best_ar is None or aicc < best_ar["AICc"]:
                best_ar = {"p": p, "fit": fit, "AIC": fit.aic, "AICc": aicc, "BIC": fit.bic, "k": k}
        except Exception:
            continue
    grids["AR"] = pd.DataFrame(ar_rows).sort_values("AICc")
    if best_ar:
        pred = np.asarray(best_ar["fit"].forecast(HORIZON))
        name = f"AR({best_ar['p']})"
        record(results, name, f"AR({best_ar['p']}) dengan konstanta", best_ar["fit"], pred, train,
test, best_ar["k"], best_ar["AIC"], best_ar["AICc"], best_ar["BIC"])
        fitted[name] = pred
        phi = np.asarray(best_ar["fit"].params[1:], dtype=float)
        root_rows.append(root_summary(name, np.roots(np.r_[-phi[::-1], 1.0]), "AR"))

    def grid_search(d, max_pq=3, label="ARMA"):
        rows = []
        best = None
        for p in range(max_pq + 1):
            for q in range(max_pq + 1):
                if p == 0 and q == 0:
                    continue
                try:
```

```
                    fit = SARIMAX(
                        train, order=(p, d, q), trend="c" if d == 0 else "n",
                        enforce_stationarity=False, enforce_invertibility=False,
                    ).fit(disp=False)
                    if not np.isfinite(fit.aicc):
                        continue
                    rows.append(
                        {
                            "p": p, "d": d, "q": q, "AIC": fit.aic, "AICc": fit.aicc,
                            "BIC": fit.bic, "k": len(fit.params),
                            "LB20_p": lb_pvalue(fit.resid),
                            "Konvergen": bool(fit.mle_retvals.get("converged", True)),
                        }
                    )
                    if best is None or fit.aicc < best["AICc"]:
                        best = {"p": p, "d": d, "q": q, "fit": fit, "AIC": fit.aic, "AICc": fit.aicc,
"BIC": fit.bic, "k": len(fit.params)}
                except Exception:
                    continue
        grids[label] = pd.DataFrame(rows).sort_values("AICc")
        return best

    best_arma = grid_search(0, 3, "ARMA")
    if best_arma:
        pred = np.asarray(best_arma["fit"].forecast(HORIZON))
        name = f"ARMA({best_arma['p']},{best_arma['q']})"
        record(results, name, f"ARMA({best_arma['p']},{best_arma['q']})", best_arma["fit"], pred,
train, test, best_arma["k"], best_arma["AIC"], best_arma["AICc"], best_arma["BIC"])
        fitted[name] = pred
        root_rows.append(root_summary(name, best_arma["fit"].arroots, "AR"))
        root_rows.append(root_summary(name, best_arma["fit"].maroots, "MA"))

    best_arima = grid_search(1, 3, "ARIMA")
    if best_arima:
        pred = np.asarray(best_arima["fit"].forecast(HORIZON))
        name = f"ARIMA({best_arima['p']},1,{best_arima['q']})"
        record(results, name, f"ARIMA({best_arima['p']},1,{best_arima['q']})", best_arima["fit"], pred,
train, test, best_arima["k"], best_arima["AIC"], best_arima["AICc"], best_arima["BIC"])
        fitted[name] = pred

    sarima_rows = []
    best_sarima = None
    for p in range(3):
        for q in range(3):
            for P in range(2):
                for Q in range(2):
                    try:
                        fit = SARIMAX(
                            train, order=(p, 1, q), seasonal_order=(P, 0, Q, SEASON), trend="n",
                            enforce_stationarity=False, enforce_invertibility=False,
                        ).fit(disp=False)
                        if not np.isfinite(fit.aicc) or getattr(fit, "params",
pd.Series()).get("sigma2", 1) <= 0:
                            continue
                        row = {
                            "p": p, "d": 1, "q": q, "P": P, "D": 0, "Q": Q, "s": SEASON,
                            "AIC": fit.aic, "AICc": fit.aicc, "BIC": fit.bic, "k": len(fit.params),
                            "LB20_p": lb_pvalue(fit.resid),
                            "Konvergen": bool(fit.mle_retvals.get("converged", True)),
                        }
                        sarima_rows.append(row)
                        if row["Konvergen"] and row["LB20_p"] > 0.05:
                            if best_sarima is None or fit.aicc < best_sarima["AICc"]:
                                best_sarima = {**row, "fit": fit}
                    except Exception:
                        continue
    grids["SARIMA"] = pd.DataFrame(sarima_rows).sort_values("AICc")
    if best_sarima is None:
        best_sarima = {
            "p": FINAL_ORDER[0], "q": FINAL_ORDER[2], "P": FINAL_SEASONAL[0], "Q": FINAL_SEASONAL[2],
            "fit": SARIMAX(train, order=FINAL_ORDER, seasonal_order=FINAL_SEASONAL, trend="n",
                           enforce_stationarity=False, enforce_invertibility=False).fit(disp=False),
        }
        best_sarima["AIC"] = best_sarima["fit"].aic
        best_sarima["AICc"] = best_sarima["fit"].aicc
        best_sarima["BIC"] = best_sarima["fit"].bic
```

```
        best_sarima["k"] = len(best_sarima["fit"].params)
    pred = np.asarray(best_sarima["fit"].forecast(HORIZON))
    sarima_name =
f"SARIMA({best_sarima['p']},1,{best_sarima['q']})({best_sarima['P']},0,{best_sarima['Q']})[{SEASON}]"
    record(results, sarima_name, "Model musiman terpilih (AICc + Ljung-Box)", best_sarima["fit"], pred,
train, test, best_sarima["k"], best_sarima["AIC"], best_sarima["AICc"], best_sarima["BIC"])
    fitted[sarima_name] = pred
    root_rows.append(root_summary(sarima_name, best_sarima["fit"].arroots, "AR"))
    root_rows.append(root_summary(sarima_name, best_sarima["fit"].maroots, "MA"))

    d1_rows = []
    for order, seasonal in [((1, 1, 0), (1, 1, 0, SEASON)), ((1, 1, 1), (1, 1, 0, SEASON)), ((0, 1, 1),
(1, 1, 0, SEASON))]:
        try:
            fit = SARIMAX(train, order=order, seasonal_order=seasonal, trend="n",
                          enforce_stationarity=False, enforce_invertibility=False).fit(disp=False)
            pred_d1 = np.asarray(fit.forecast(HORIZON))
            d1_rows.append(
                {
                    "Model": f"SARIMA{order}({seasonal[0]},{seasonal[1]},{seasonal[2]})[{SEASON}]",
                    **score(test.values, pred_d1, train),
                    "AIC": fit.aic, "AICc": fit.aicc, "LB20_p": lb_pvalue(fit.resid),
                }
            )
        except Exception:
            continue
    grids["SARIMA_D1"] = pd.DataFrame(d1_rows)

    def fourier_matrix(t, k, period=SEASON):
        return np.column_stack(
            [func(2 * np.pi * j * t / period) for j in range(1, k + 1) for func in (np.sin, np.cos)]
        )

    fourier_rows = []
    best_fourier = None
    for k in (1, 2, 3):
        for order in ((1, 1, 0), (1, 1, 1), (2, 1, 0)):
            try:
                X_train = fourier_matrix(np.arange(len(train)), k)
                X_test = fourier_matrix(np.arange(len(train), len(train) + HORIZON), k)
                fit = SARIMAX(train, exog=X_train, order=order, trend="n",
                              enforce_stationarity=False, enforce_invertibility=False).fit(disp=False)
                pred = np.asarray(fit.forecast(HORIZON, exog=X_test))
                row = {"K": k, "p": order[0], "d": order[1], "q": order[2],
                       "AIC": fit.aic, "AICc": fit.aicc, "BIC": fit.bic,
                       **score(test.values, pred, train), "LB20_p": lb_pvalue(fit.resid)}
                fourier_rows.append(row)
                if k == 2 and order == (1, 1, 1):
                    best_fourier = {"k": k, "order": order, "fit": fit, "pred": pred, "X_train":
X_train, "X_test": X_test, "row": row}
            except Exception:
                continue
    grids["Fourier"] = pd.DataFrame(fourier_rows).sort_values("RMSE")
    if best_fourier:
        name = "Fourier K=2 + ARIMA(1,1,1)"
        record(results, name, "Regresi Fourier (K=2) + ARIMA(1,1,1)", best_fourier["fit"],
best_fourier["pred"], train, test, len(best_fourier["fit"].params), best_fourier["row"]["AIC"],
best_fourier["row"]["AICc"], best_fourier["row"]["BIC"], "Bonus")
        fitted[name] = best_fourier["pred"]

    comparison = pd.DataFrame(results)
    comparison = comparison.sort_values(["RMSE", "AICc"], na_position="last").reset_index(drop=True)
    return comparison, fitted, grids["ARMA"], grids["ARIMA"], grids["SARIMA"], {
        "best_sarima": best_sarima,
        "fourier": best_fourier,
        "grids": grids,
        "es_params": pd.DataFrame(es_rows),
        "roots": pd.DataFrame([row for row in root_rows if row is not None]),
    }


def fit_final(series: pd.Series):
    fit = SARIMAX(
        series, order=FINAL_ORDER, seasonal_order=FINAL_SEASONAL, trend="n",
        enforce_stationarity=False, enforce_invertibility=False,
    ).fit(disp=False)
```

```
    forecast_result = fit.get_forecast(HORIZON)
    forecast = forecast_result.predicted_mean
    ci80 = forecast_result.conf_int(alpha=0.20)
    ci95 = forecast_result.conf_int(alpha=0.05)
    forecast_frame = pd.DataFrame(
        {
            "Minggu": pd.date_range(series.index[-1] + pd.Timedelta(days=7), periods=HORIZON,
freq="7D"),
            "Peramalan": forecast.values,
            "Batas_bawah_80": ci80.iloc[:, 0].values,
            "Batas_atas_80": ci80.iloc[:, 1].values,
            "Batas_bawah_95": ci95.iloc[:, 0].values,
            "Batas_atas_95": ci95.iloc[:, 1].values,
        }
    )
    coefficients = pd.DataFrame(
        {
            "Parameter": fit.params.index,
            "Koefisien": fit.params.values,
            "Std_Error": fit.bse.values,
            "z": fit.tvalues.values,
            "p_value": fit.pvalues.values,
            "CI_bawah": fit.conf_int().iloc[:, 0].values,
            "CI_atas": fit.conf_int().iloc[:, 1].values,
        }
    )
    resid = fit.resid
    lb = acorr_ljungbox(resid, lags=[10, 20, 52], return_df=True)
    jb_stat, jb_p, jb_skew, jb_kurt = jarque_bera(resid)
    arch_stat, arch_p, _, _ = het_arch(resid, nlags=5)
    shapiro_stat, shapiro_p = stats.shapiro(resid)
    diagnostics = pd.DataFrame(
        [
            ("Ljung-Box lag 10", lb["lb_stat"].iloc[0], lb["lb_pvalue"].iloc[0], "Tidak ada
autokorelasi" if lb["lb_pvalue"].iloc[0] > 0.05 else "Ada autokorelasi"),
            ("Ljung-Box lag 20", lb["lb_stat"].iloc[1], lb["lb_pvalue"].iloc[1], "Tidak ada
autokorelasi" if lb["lb_pvalue"].iloc[1] > 0.05 else "Ada autokorelasi"),
            ("Ljung-Box lag 52", lb["lb_stat"].iloc[2], lb["lb_pvalue"].iloc[2], "Tidak ada
autokorelasi" if lb["lb_pvalue"].iloc[2] > 0.05 else "Ada autokorelasi"),
            ("Jarque-Bera", jb_stat, jb_p, "Normal" if jb_p > 0.05 else "Tidak normal"),
            ("ARCH LM (5 lag)", arch_stat, arch_p, "Homoskedastik" if arch_p > 0.05 else
"Heteroskedastik"),
            ("Shapiro-Wilk", shapiro_stat, shapiro_p, "Normal" if shapiro_p > 0.05 else "Tidak
normal"),
            ("Durbin-Watson", durbin_watson(resid), np.nan, "Mendekati 2 = baik"),
        ],
        columns=["Uji", "Statistik", "p_value", "Interpretasi"],
    )
    return fit, forecast_frame, coefficients, diagnostics, resid


def plot_model_rmse(comparison: pd.DataFrame) -> None:
    key = comparison[~comparison["Model"].isin(["Seasonal naive", "MA(3)", "MA(8)"])].copy()
    key = key.sort_values("RMSE", ascending=True)
    fig, ax = plt.subplots(figsize=(9.5, 5.4))
    bars = ax.barh(key["Model"], key["RMSE"], color=COLORS["blue"], alpha=0.75)
    for bar_, value in zip(bars, key["RMSE"]):
        ax.text(value + 0.08, bar_.get_y() + bar_.get_height() / 2, f"{value:.2f}", va="center",
fontsize=8)
    ax.invert_yaxis()
    ax.set_xlabel("RMSE pada data uji (µg/m³)")
    ax.set_title("Perbandingan RMSE model pada 12 minggu uji")
    fig.tight_layout()
    fig.savefig(FIG / "fig06_model_rmse.png", bbox_inches="tight")
    plt.close(fig)


def plot_test_forecasts(test: pd.Series, fitted: dict) -> None:
    selected = [
        "Naive (random walk)",
        "Holt",
        "ARIMA(1,1,3)",
        FINAL_NAME,
        "Fourier K=2 + ARIMA(1,1,1)",
    ]
    available = [name for name in selected if name in fitted]
```

```
    fig, ax = plt.subplots(figsize=(10, 4.8))
    ax.plot(test.index, test.values, color="black", marker="o", linewidth=2, label="Aktual")
    colors = [COLORS["gray"], COLORS["orange"], COLORS["green"], COLORS["blue"], COLORS["purple"]]
    for name, color in zip(available, colors):
        ax.plot(test.index, fitted[name], marker="s", ms=3, linewidth=1.4, linestyle="--", color=color,
label=name)
    ax.set_title("Aktual vs peramalan pada periode uji (12 minggu)")
    ax.set_ylabel("PM2.5 (µg/m³)")
    ax.set_xlabel("Tanggal")
    ax.legend(fontsize=8, ncol=2)
    fig.tight_layout()
    fig.savefig(FIG / "fig07_test_forecasts.png", bbox_inches="tight")
    plt.close(fig)


def plot_residual_diagnostics(resid: pd.Series, fit) -> None:
    fig, axes = plt.subplots(2, 2, figsize=(11, 7))
    axes[0, 0].plot(resid.index, resid.values, color=COLORS["gray"], marker="o", ms=2, linewidth=0.8)
    axes[0, 0].axhline(0, color="black", linewidth=0.8)
    axes[0, 0].set_title("Residu terstandarisasi")
    plot_acf(resid, lags=40, ax=axes[0, 1], color=COLORS["blue"])
    axes[0, 1].set_title("ACF residu")
    axes[1, 0].hist(resid, bins=18, color=COLORS["blue"], alpha=0.7, edgecolor="white", density=True)
    x_grid = np.linspace(resid.min(), resid.max(), 200)
    axes[1, 0].plot(x_grid, stats.norm.pdf(x_grid, resid.mean(), resid.std()), color=COLORS["red"],
linewidth=1.5)
    axes[1, 0].set_title("Histogram residu")
    stats.probplot(resid, dist="norm", plot=axes[1, 1])
    axes[1, 1].set_title("QQ-plot residu")
    fig.tight_layout()
    fig.savefig(FIG / "fig08_residual_diagnostics.png", bbox_inches="tight")
    plt.close(fig)


def plot_forecast(series: pd.Series, forecast_frame: pd.DataFrame, train: pd.Series, test: pd.Series) -
> None:
    fig, ax = plt.subplots(figsize=(11, 5))
    ax.plot(series.index, series.values, color=COLORS["blue"], linewidth=1.1, label="Histori")
    ax.axvspan(train.index[0], train.index[-1], color=COLORS["green"], alpha=0.06)
    ax.axvspan(test.index[0], test.index[-1], color=COLORS["red"], alpha=0.08, label="Periode uji")
    ax.plot(forecast_frame["Minggu"], forecast_frame["Peramalan"], color=COLORS["red"], marker="o",
ms=3, linewidth=1.8, label="Peramalan")
    ax.fill_between(
        forecast_frame["Minggu"], forecast_frame["Batas_bawah_95"], forecast_frame["Batas_atas_95"],
        color=COLORS["red"], alpha=0.12, label="CI 95%",
    )
    ax.fill_between(
        forecast_frame["Minggu"], forecast_frame["Batas_bawah_80"], forecast_frame["Batas_atas_80"],
        color=COLORS["red"], alpha=0.22, label="CI 80%",
    )
    ax.axvline(series.index[-1], color=COLORS["gray"], linestyle="--", linewidth=1)
    ax.set_title("Peramalan PM2.5 mingguan 12 minggu ke depan")
    ax.set_ylabel("PM2.5 (µg/m³)")
    ax.set_xlabel("Tanggal")
    ax.legend(loc="upper left", ncol=5)
    fig.tight_layout()
    fig.savefig(FIG / "fig09_forecast.png", bbox_inches="tight")
    plt.close(fig)


def dm_test(e1: np.ndarray, e2: np.ndarray) -> tuple[float, float]:
    d = np.asarray(e1) ** 2 - np.asarray(e2) ** 2
    n = len(d)
    dbar = float(np.mean(d))
    var = float(np.var(d, ddof=1) / n)
    if var <= 0:
        return np.nan, np.nan
    stat = dbar / np.sqrt(var)
    p_value = 2 * (1 - stats.t.cdf(abs(stat), df=n - 1))
    return float(stat), float(p_value)


def style_workbook(path: Path, sheets: dict[str, pd.DataFrame], notes: list[str]) -> None:
    from openpyxl import load_workbook
    from openpyxl.styles import Alignment, Font, PatternFill
    from openpyxl.utils import get_column_letter
```

```
    with pd.ExcelWriter(path, engine="openpyxl") as writer:
        summary = pd.DataFrame(
            [
                ("Sumber data", "Open-Meteo Air Quality API (CAMS global)"),
                ("Lokasi", "Kota Surakarta (-7,5545; 110,8060)"),
                ("Frekuensi", "Mingguan (rata-rata 168 jam)"),
                ("Periode model", "2022-08-08 s.d. 2024-10-21"),
                ("Jumlah observasi", len(sheets.get("Data_Mingguan", pd.DataFrame()))),
                ("Model final", FINAL_NAME),
            ],
            columns=["Keterangan", "Nilai"],
        )
        summary.to_excel(writer, sheet_name="Ringkasan", index=False)
        for name, frame in sheets.items():
            frame.to_excel(writer, sheet_name=name, index=False)
        pd.DataFrame({"Catatan": notes}).to_excel(writer, sheet_name="Catatan", index=False)

    workbook = load_workbook(path)
    header_fill = PatternFill("solid", fgColor="1F4E79")
    header_font = Font(color="FFFFFF", bold=True)
    for worksheet in workbook.worksheets:
        worksheet.freeze_panes = "A2"
        for cell in worksheet[1]:
            cell.fill = header_fill
            cell.font = header_font
            cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
        for column_cells in worksheet.columns:
            length = max(len(str(cell.value)) if cell.value is not None else 0 for cell in
column_cells)
            worksheet.column_dimensions[get_column_letter(column_cells[0].column)].width = min(max(12,
length + 2), 55)
        for row in worksheet.iter_rows(min_row=2):
            for cell in row:
                if isinstance(cell.value, float):
                    cell.number_format = "0.000"
    workbook.save(path)


def main() -> None:
    frame, series = load_series()
    train, test = series.iloc[:-TEST_SIZE], series.iloc[-TEST_SIZE:]
    print(f"Observasi: {len(series)} | Train: {len(train)} | Test: {len(test)}")
    print(f"Periode: {series.index.min().date()} s.d. {series.index.max().date()}")

    build_flowchart()
    desc = descriptive(series)
    stationarity = stationarity_table(series)
    plot_series(series, train, test)
    seasonal_strength = plot_decomposition(series)
    plot_seasonal_profile(series)
    plot_acf_pacf(series)

    comparison, fitted, arma_grid, arima_grid, sarima_grid, extra = fit_all_models(train, test)
    best_sarima = extra["best_sarima"]
    sarima_name =
f"SARIMA({best_sarima['p']},1,{best_sarima['q']})({best_sarima['P']},0,{best_sarima['Q']})[{SEASON}]"
    print(f"SARIMA terpilih: {sarima_name} | AICc={best_sarima['AICc']:.2f}")

    plot_model_rmse(comparison)
    plot_test_forecasts(test, fitted)

    final_fit, forecast_frame, coefficients, diagnostics, resid = fit_final(series)
    plot_residual_diagnostics(resid, final_fit)
    plot_forecast(series, forecast_frame, train, test)

    prediction_errors = {
        row["Model"]: test.values - row["_forecast"] for _, row in comparison.iterrows()
    }
    dm_rows = []
    final_model_name = comparison.loc[comparison["AICc"].idxmin(), "Model"] if
comparison["AICc"].notna().any() else sarima_name
    final_model_name = sarima_name if sarima_name in prediction_errors else final_model_name
    for name, errors in prediction_errors.items():
        if name == final_model_name:
            continue
```

```
    stat, p_value = dm_test(prediction_errors[final_model_name], errors)
    dm_rows.append(
        {
            "Model pembanding": name,
            "DM statistik": stat,
            "p-value": p_value,
            "Kesimpulan": "Tidak berbeda signifikan" if p_value > 0.05 else "Berbeda signifikan",
        }
    )
dm_table = pd.DataFrame(dm_rows)

compact = comparison.drop(columns=["_forecast"]).copy()
compact.to_csv(TAB / "model_comparison.csv", index=False)
desc.to_csv(TAB / "descriptive.csv", index=False)
stationarity.to_csv(TAB / "stationarity.csv", index=False)
coefficients.to_csv(TAB / "final_coefficients.csv", index=False)
diagnostics.to_csv(TAB / "final_diagnostics.csv", index=False)
forecast_frame.to_csv(TAB / "forecast.csv", index=False)
dm_table.to_csv(TAB / "dm_tests.csv", index=False)
arma_grid.to_csv(TAB / "grid_arma.csv", index=False)
arima_grid.to_csv(TAB / "grid_arima.csv", index=False)
sarima_grid.to_csv(TAB / "grid_sarima.csv", index=False)
extra["grids"]["Fourier"].to_csv(TAB / "grid_fourier.csv", index=False)
extra["grids"]["SARIMA_D1"].to_csv(TAB / "sarima_d1.csv", index=False)
extra["grids"]["AR"].to_csv(TAB / "grid_ar.csv", index=False)
extra["es_params"].to_csv(TAB / "es_params.csv", index=False)
extra["roots"].to_csv(TAB / "roots.csv", index=False)

monthly_context = pd.read_csv(DATA / "pm25_surakarta_monthly.csv")
pm_ratio = pd.read_csv(DATA / "pm25_pm10_monthly.csv")
monthly_context.to_csv(TAB / "monthly_context.csv", index=False)
pm_ratio.to_csv(TAB / "pm_ratio.csv", index=False)

forecast_first = float(forecast_frame["Peramalan"].iloc[0])
forecast_last = float(forecast_frame["Peramalan"].iloc[-1])
forecast_mean = float(forecast_frame["Peramalan"].mean())
final_row = compact.loc[compact["Model"] == final_model_name].iloc[0]
lb_row = diagnostics.set_index("Uji")["p_value"]

summary = {
    "n": int(len(series)),
    "n_train": int(len(train)),
    "n_test": int(len(test)),
    "start": series.index.min().strftime("%Y-%m-%d"),
    "end": series.index.max().strftime("%Y-%m-%d"),
    "mean": float(series.mean()),
    "std": float(series.std()),
    "min": float(series.min()),
    "max": float(series.max()),
    "seasonal_strength": seasonal_strength,
    "final_model": final_model_name,
    "final_spec": f"SARIMA{FINAL_ORDER}({FINAL_SEASONAL[0]},0,{FINAL_SEASONAL[2]})[{SEASON}]",
    "aic": float(final_fit.aic),
    "aicc": float(final_fit.aicc),
    "bic": float(final_fit.bic),
    "loglik": float(final_fit.llf),
    "sigma2": float(final_fit.params.get("sigma2", np.nan)),
    "test_mae": float(final_row["MAE"]),
    "test_rmse": float(final_row["RMSE"]),
    "test_mape": float(final_row["MAPE"]),
    "test_smape": float(final_row["sMAPE"]),
    "test_mase": float(final_row["MASE"]),
    "lb10_p": float(lb_row.get("Ljung-Box lag 10", np.nan)),
    "lb20_p": float(lb_row.get("Ljung-Box lag 20", np.nan)),
    "lb52_p": float(lb_row.get("Ljung-Box lag 52", np.nan)),
    "jb_p": float(lb_row.get("Jarque-Bera", np.nan)),
    "arch_p": float(lb_row.get("ARCH LM (5 lag)", np.nan)),
    "shapiro_p": float(lb_row.get("Shapiro-Wilk", np.nan)),
    "forecast_first": forecast_first,
    "forecast_last": forecast_last,
    "forecast_mean": forecast_mean,
    "forecast": forecast_frame.to_dict(orient="records"),
    "sarima_coef": coefficients.to_dict(orient="records"),
    "diagnostics": diagnostics.to_dict(orient="records"),
    "dm": dm_table.to_dict(orient="records"),
    "comparison": compact.to_dict(orient="records"),
```

```
        "stationarity": stationarity.to_dict(orient="records"),
        "descriptive": desc.to_dict(orient="records"),
        "pm_ratio_pre": float(pm_ratio.loc[pm_ratio["homogeneous"], "ratio"].mean()),
        "pm_ratio_post": float(pm_ratio.loc[~pm_ratio["homogeneous"], "ratio"].mean()),
        "break": "2024-11-01",
    }
    with (OUT / "results.json").open("w", encoding="utf-8") as handle:
        json.dump(summary, handle, indent=2, ensure_ascii=False, default=to_py)

    sheets = {
        "Data_Mingguan": frame,
        "Statistik_Deskriptif": desc,
        "Uji_Stasioneritas": stationarity,
        "Perbandingan_Model": compact,
        "Grid_AR": extra["grids"]["AR"],
        "Grid_ARMA": arma_grid,
        "Grid_ARIMA": arima_grid,
        "Grid_SARIMA": sarima_grid,
        "Grid_Fourier": extra["grids"]["Fourier"],
        "SARIMA_D1": extra["grids"]["SARIMA_D1"],
        "Parameter_ES": extra["es_params"],
        "Akar_Model": extra["roots"],
        "Koefisien_Final": coefficients,
        "Diagnostik_Final": diagnostics,
        "Peramalan": forecast_frame,
        "Uji_DM": dm_table,
        "Bulanan_Konteks": monthly_context,
        "Rasio_PM": pm_ratio,
    }
    notes = [
        "AIC/AICc/BIC hanya sebanding untuk model dengan orde diferensiasi (d) dan transformasi data
yang sama.",
        "AIC pada SES/Holt/Holt-Winters tidak sebanding langsung dengan model ARIMA; kolom
AIC_sebanding menandai hal ini.",
        "Periode model berhenti Oktober 2024 karena patahan model CAMS pada November 2024 (rasio
PM10/PM2.5 turun dari ~1,46 menjadi ~1,04).",
        "Metrik uji dihitung pada 12 minggu terakhir (Agustus–Oktober 2024); model dilatih pada 104
minggu.",
        "MASE memakai seasonal naive (lag 52) pada data latih sebagai skala.",
        "Interval peramalan disajikan pada tingkat kepercayaan 80% dan 95%.",
        "Parameter_ES memuat parameter smoothing dan nilai awal hasil estimasi statsmodels.",
        "Akar_Model memuat modulus akar AR/MA; nilai > 1 menunjukkan stasioneritas/invertibilitas.",
    ]
    style_workbook(OUT / "Hasil_Analisis_PM25_Surakarta.xlsx", sheets, notes)

    print("\nPerbandingan model (ringkas):")
    print(compact[["Model", "AICc", "RMSE", "MAE", "MAPE", "MASE",
"LB20_p"]].head(15).round(3).to_string(index=False))
    print("\nDiagnostik final:")
    print(diagnostics.round(4).to_string(index=False))
    print("\nPeramalan:")
    print(forecast_frame.round(2).to_string(index=False))
    print(f"\nOutput: {OUT}")


if __name__ == "__main__":
    main()
```

LAMPIRAN C. KELUARAN TAMBAHAN

Seluruh keluaran numerik tersedia di folder outputs/. Workbook Excel memuat sheet Ringkasan, Data_Mingguan, Statistik_Deskriptif, Uji_Stasioneritas, Perbandingan_Model, Grid_AR, Grid_ARMA, Grid_ARIMA, Grid_SARIMA, Grid_Fourier, SARIMA_D1, Parameter_ES, Akar_Model, Koefisien_Final, Diagnostik_Final, Peramalan, Uji_DM, Bulanan_Konteks, Rasio_PM, dan Catatan.

Tabel 16. Daftar berkas keluaran utama

Berkas	Isi
outputs/results.json	Ringkasan angka analisis dalam format JSON
outputs/Hasil_Analisis_PM25_Surakarta.xlsx	Workbook Excel seluruh tabel
outputs/figures/fig00–fig09	Gambar alur, data, dekomposisi, ACF/PACF, model, diagnostik, peramalan
outputs/tables/*.csv	Tabel statistik, grid kandidat, koefisien, diagnostik, peramalan
notebooks/01_analisis_pm25_surakarta.ipynb	Walkthrough reproducible dengan output inline

Sumber: Penulis (2026).

LAMPIRAN D. PERNYATAAN PENGGUNAAN ALAT BANTU AI

Alat bantu AI digunakan untuk membantu penulisan kode, merapikan struktur laporan, dan memeriksa konsistensi angka. Seluruh keputusan metodologis, pemilihan model, interpretasi hasil, dan verifikasi angka dilakukan oleh penulis. Penulis bertanggung jawab penuh atas kebenaran isi laporan dan memastikan tidak ada data yang direkayasa.